

arista de menor costo en *aristas* lleva un tiempo $O(\log a)$. Así, las operaciones en la cola de prioridad llevan un tiempo $O(\log a)$ en el peor caso. El tiempo total requerido para realizar las operaciones COMBINA y ENCUENTRA depende del método usado para implantar el CONJUNTO_CE. Como se muestra en la sección 5.5, hay métodos $O(\log a)$ y $O(\alpha(a))$. En cualquiera de los casos, el algoritmo de Kruskal se puede implantar para que se ejecute en tiempo $O(\log a)$.

7.3 Recorridos

En un gran número de problemas con grafos, es necesario visitar sistemáticamente los vértices del grafo. Las búsquedas en profundidad y en amplitud, temas de esta sección, son dos técnicas importantes para hacerlo. Ambas técnicas pueden usarse para determinar de manera eficiente todos los vértices que están conectados a un vértice dado.

```

procedure Kruskal ( V: CONJUNTO de vértices;
                    A: CONJUNTO de aristas;
                    var T: CONJUNTO de aristas );
var
    comp_n: integer; { número actual de componentes }
    aristas: COLA_DE_PRIORIDAD; { el conjunto de aristas }
    componentes: CONJUNTO_CE; { el conjunto V agrupado en
                                un conjunto de componentes COMBINA_ENCUESTRA }
    u, v: vértice;
    a: arista;
    comp_siguiente: integer; { nombre para el nuevo componente }
    comp_u, comp_v; { nombres de componentes }

begin
    ANULA(T);
    ANULA(aristas);
    comp_siguiente := 0;
    comp_n := número de miembros de V;
    for v en V do begin { asigna valor inicial a un componente
                          para que contenga un vértice de V }
        comp_siguiente := comp_siguiente + 1;
        INICIAL(comp_siguiente, v, componentes)
    end;
    for a en A do { asigna valor inicial a la cola de prioridad de aristas }
        INSERTA(a, aristas);
    while comp_n > 1 do begin { considera la siguiente arista }
        a := SUPRIME_MIN(aristas)
        sea a = (u, v);
        comp_u := ENCUENTRA(u, componentes);
        comp_v := ENCUENTRA(v, componentes);

```

```

    if comp_u <> comp_v then begin
      { a conecta dos componentes diferentes }
      COMBINA(comp_u, comp_v, componentes);
      comp_n := comp_n - 1;
      INSERTA(a, T)
    end
  end
end; { Kruskal }

```

Fig. 7.10. Algoritmo de Kruskal.

Búsqueda en profundidad

Recuérdese de la sección 6.5 el algoritmo *bpf* para búsquedas en grafos dirigidos. El mismo algoritmo puede emplearse para búsqueda en grafos no dirigidos, puesto que la arista no dirigida (v, w) puede considerarse como el par de aristas dirigidas $v \rightarrow w$ y $w \rightarrow v$.

De hecho, los bosques abarcadores en profundidad, construidos para grafos no dirigidos, son más simples que para los dirigidos. Primero, se debe observar que cada árbol del bosque es un componente conexo del grafo, y que si el grafo fuera conexo, tendría sólo un árbol en su bosque. Segundo, para grafos dirigidos se identifican cuatro clases de arcos: de árbol, de avance, de retroceso y cruzado. Para grafos no dirigidos sólo hay dos clases: aristas de árbol y de retroceso.

Dado que en grafos no dirigidos no existe distinción entre las aristas de retroceso y las de avance, se denominarán arcos *de retroceso*. En un grafo no dirigido no existen las aristas cruzadas, esto es, aristas (v, w) donde v no es antecesor ni descendiente de w en el árbol abarcador. Supóngase que las hubiera; entonces, sea v un vértice alcanzado antes que w en la búsqueda. La llamada a *bpf*(v) no puede terminar hasta haber buscado w , así que w se introduce en el árbol como descendiente de v . De modo semejante, si *bpf*(w) se llama antes que *bpf*(v), v se convierte en descendiente de w .

Como resultado, durante una búsqueda en profundidad en un grafo no dirigido G , todas las aristas pueden ser,

1. *aristas de árbol*, aquellas aristas (v, w) tales que *bpf*(v) llama directamente a *bpf*(w) o viceversa, o bien
2. *aristas de retroceso*, aquellas aristas (v, w) tales que ni *bpf*(v) ni *bpf*(w) se llaman directamente, pero una llamó indirectamente a la otra (es decir, *bpf*(w) llama a *bpf*(x), que llama a *bpf*(v), de modo que w es antecesor de v).

Ejemplo 7.6. Considérese el grafo conexo G de la figura 7.11(a). Un árbol abarcador en profundidad T resultante de una búsqueda en profundidad de G se muestra en la figura 7.11(b). Se supuso que la búsqueda empezó en el vértice a , y se adoptó la convención de mostrar las aristas del árbol con líneas de trazo continuo y las aristas de retroceso con líneas de puntos. El árbol se dibujó con la raíz en la parte

superior, y los hijos de cada vértice, en el orden de izquierda a derecha en que fueron visitados por el procedimiento *bpfl*.

Para seguir unos cuantos pasos de la búsqueda, el procedimiento *bpfl(a)* llama a *bpfl(b)* y añade la arista (a, b) a T , ya que b no ha sido visitado. En b , *bpfl* llama a *bpfl(d)* y agrega la arista (b, d) a T . En d , *bpfl* llama a *bpfl(e)* y añade la arista (d, e) a T . En e , los vértices a , b y d ya están marcados como visitados, de modo que *bpfl(e)* regresa sin incorporar ninguna arista a T . En d , *bpfl* encuentra los vértices a y b marcados como visitados, así que *bpfl(d)* regresa también sin agregar más aristas a T . En b , *bpfl* encuentra los vértices adyacentes restantes a y e marcados como visitados, así que *bpfl(b)* regresa. La búsqueda continúa después con c , f y g . □

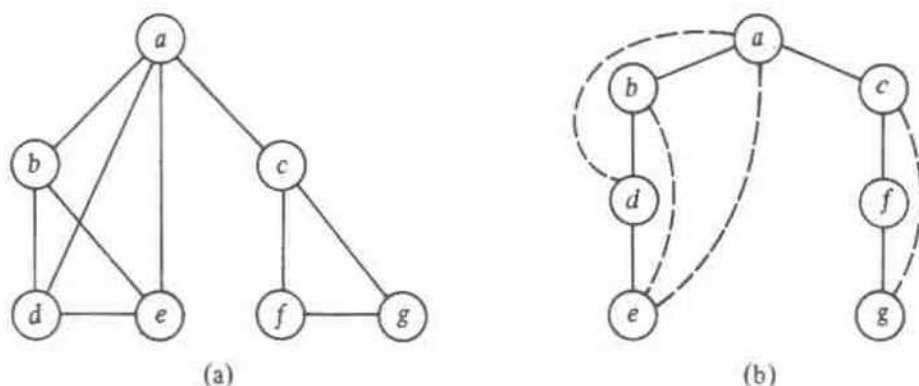


Fig. 7.11. Un grafo y su búsqueda en profundidad.

Búsqueda en amplitud

Otra forma sistemática de visitar los vértices se conoce como *búsqueda en amplitud*. Este enfoque se denomina «en amplitud» porque desde cada vértice v que se visita se busca en forma tan amplia como sea posible, visitando todos los vértices adyacentes a v . Esta estrategia de búsqueda también se puede aplicar a grafos dirigidos.

Igual que en la búsqueda en profundidad, al realizar una búsqueda en amplitud se puede construir un bosque abarcador. En este caso, se considera la arista (x, y) como una arista de árbol si el vértice y es el que se visitó primero partiendo del vértice x del ciclo interno del procedimiento de búsqueda *bea* de la figura 7.12.

Resulta que para la búsqueda en amplitud en un grafo no dirigido, toda arista que no es de árbol es una arista cruzada; esto es, conecta dos vértices ninguno de los cuales es antecesor del otro.

El algoritmo de búsqueda en amplitud de la figura 7.12 inserta las aristas de árbol en un conjunto T , que se supone inicialmente vacío. Se presume que cada entrada en el arreglo *marca* tiene asignado el valor inicial *no_visitado*; la figura 7.12 opera en un componente conexo. Si el grafo no es conexo, *bea* debe llamarse desde un vértice de cada componente. Obsérvese que en una búsqueda en amplitud se debe

marcar cada vértice como visitado antes de meterlo en la cola, y así evitar que se coloque en la cola más de una vez.

Ejemplo 7.7. El árbol abarcador en amplitud del grafo G de la figura 7.11(a) se muestra en la figura 7.13. Se supone que la búsqueda empieza en el vértice a . Como antes, las aristas de árbol se muestran con líneas de trazo continuo y las otras con líneas de puntos. También se ha dibujado la raíz del árbol en la parte superior, y los hijos, de izquierda a derecha, de acuerdo con el orden en que fueron visitados. □

```

procedure bea (v);
  { bea visita todos los vértices conectados a v usando búsqueda en
    amplitud }
  var
    C: COLA de vértice;
    x, y: vértice;
  begin
    marca[v] := visitado;
    PONE_EN_COLA(v, C);
    while not VACIA(C) do begin
      x := FRENTE(C);
      QUITA_DE_COLA (C);
      for cada vértice y adyacente a x do
        if marca[y] = no_visitado then begin
          marca[y] := visitado;
          PONE_EN_COLA(y, C);
          INSERTA((x, y), T)
        end
      end
    end
  end; { bea }

```

Fig. 7.12. Búsqueda en amplitud.

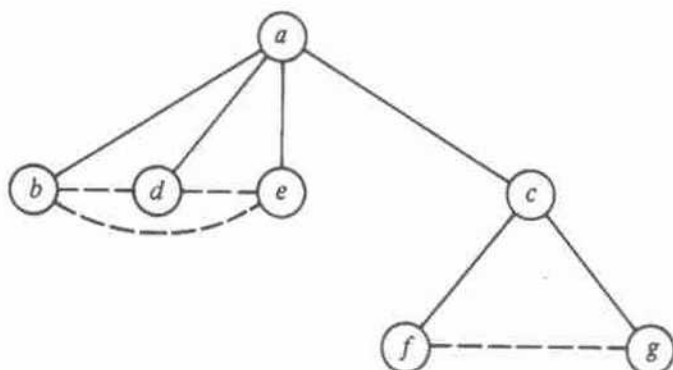


Fig. 7.13. Búsqueda en amplitud para G .

La complejidad de tiempo de la búsqueda en amplitud es la misma que para la búsqueda en profundidad. Cada vértice visitado se coloca en la cola una vez, así que el ciclo *while* se ejecuta una sola vez para cada vértice. Cada arista (x, y) se examina dos veces, desde x y desde y . Así, si el grafo tiene n vértices y a aristas, el tiempo de ejecución de *bea* es $O(\max(n, a))$ si se utiliza una representación con lista de adyacencia para las aristas. Dado que es típico que $a \geq n$, en general se hará referencia al tiempo de ejecución de la búsqueda en amplitud con $O(a)$, como ocurrió para la búsqueda en profundidad.

Las búsquedas en profundidad y en amplitud se pueden usar como marcos de trabajo, alrededor de los cuales se diseñan eficientes algoritmos para grafos. Por ejemplo, se puede emplear cualquiera de los dos métodos para encontrar los componentes conexos de un grafo, ya que aquéllos son los árboles de los dos bosques abarcadores.

Se puede verificar la existencia de ciclos por medio de la búsqueda en amplitud en un tiempo $O(n)$, donde n es el número de vértices, independientemente del número de aristas. Como se vio en la sección 7.1, cualquier grafo con n vértices y n o más aristas debe tener un ciclo. Sin embargo, un grafo puede tener $n - 1$ o menos aristas y de todos modos tener un ciclo, si tiene dos o más componentes conexos. Una forma segura de encontrar los ciclos es construir un bosque abarcador en amplitud. Así, toda arista cruzada (v, w) debe completar un ciclo simple con las aristas de árbol que conducen a v y w desde su antecesor común más cercano, como se muestra en la figura 7.14.

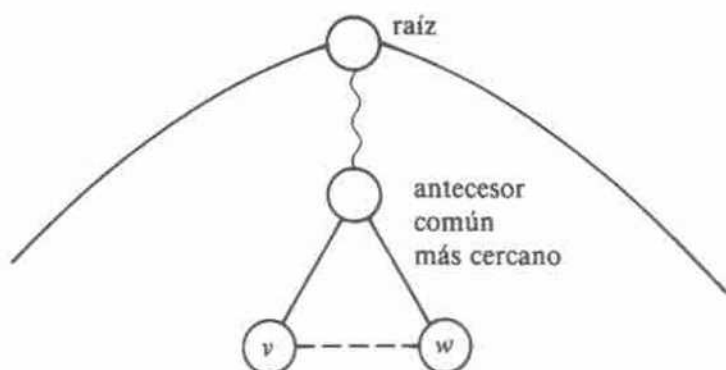


Fig. 7.14. Ciclo encontrado por la búsqueda en amplitud.

7.4 Puntos de articulación y componentes biconexos

Un *punto de articulación* de un grafo es un vértice v tal que cuando se elimina junto con todas las aristas incidentes sobre él, se divide un componente conexo en dos o más partes. Por ejemplo, los puntos de articulación de la figura 7.11(a) son a y c . Si se elimina a , el grafo, que es un componente conexo, se divide en dos triángulos: $\{b, d, e\}$ y $\{c, f, g\}$; si se elimina c , se divide en $\{a, b, d, e\}$ y $\{f, g\}$. Sin embargo, al eli-

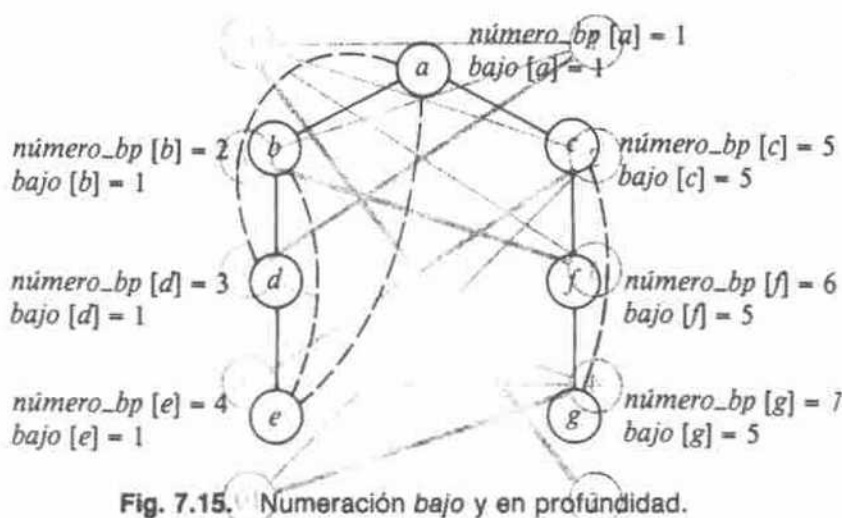
minar cualquier otro vértice del grafo de la figura 7.11(a), el componente conexo no se dividirá. A un grafo sin puntos de articulación se le llama *biconexo*. La búsqueda en profundidad es muy útil para encontrar los componentes biconexos de un grafo.

El problema de encontrar los puntos de articulación es el más simple de muchos problemas importantes relacionados con la conectividad de grafos. Como ejemplo de las aplicaciones de los algoritmos de conectividad, se puede presentar una red de comunicaciones como un grafo en el que los vértices son lugares que hay que mantener comunicados entre sí. Un grafo tiene *conectividad* k si la eliminación de $k - 1$ vértices cualesquiera no lo desconecta. Por ejemplo, un grafo tiene conectividad dos o más si, y sólo si, no tiene puntos de articulación, es decir, si, y sólo si, es biconexo. Cuanto mayor sea su conectividad, tanto más fácil será que sobreviva al fallo de alguno de sus vértices, sea por fallo de las unidades de procesamiento colocadas en los vértices o por motivos externos.

Ahora se presenta un algoritmo simple de búsqueda en profundidad para encontrar todos los puntos de articulación de un grafo, y probar por medio de su ausencia, si el grafo es biconexo.

1. Realizar una búsqueda en profundidad del grafo, calculando *número_bp*[v] para todo vértice v , como se analizó en la sección 6.5. En esencia, *número_bp* ordena los vértices como en un recorrido en orden previo del árbol abarcador en profundidad.
2. Para cada vértice v , obtener *bajo*[v], que es el *número_bp* más pequeño de v o de cualquier otro vértice w accesible desde v , siguiendo cero o más aristas de árbol hasta un descendiente x de v (x puede ser v) y después seguir una arista de retroceso (x, w). Se calcula *bajo*[v] para todos los vértices v , visitándolos en un recorrido en orden posterior. Cuando se procesa v , se ha calculado *bajo*[y] para todo hijo y de v . Se toma *bajo*[v] como el mínimo de
 - a) *número_bp*[v],
 - b) *número_bp*[z] para cualquier vértice z para el cual haya una arista de retroceso (v, z), y
 - c) *bajo*[y] para cualquier hijo y de v .
3. Ahora se encuentran los puntos de articulación como sigue.
 - a) La raíz es un punto de articulación si, y sólo si, tiene dos o más hijos. Puesto que no hay aristas cruzadas, la eliminación de la raíz debe desconectar los subárboles cuyas raíces se encuentren en sus hijos, como a desconecta $\{b, d, e\}$ de $\{c, f, g\}$ en la figura 7.11(b).
 - b) Un vértice v distinto de la raíz es un punto de articulación si, y sólo si, hay un hijo w de v tal que *bajo*[w] $\geq$ *número_bp*[v]. En este caso, v desconecta w y sus descendientes del resto del grafo. A la inversa, si *bajo*[w] $<$ *número_bp*[v], debe haber un camino para descender desde w en el árbol y regresar hasta un antecesor propio de v (el vértice cuyo *número_bp* es *bajo*[w]) y, por tanto, la eliminación de v no desconecta w ni sus descendientes del resto del grafo.

Ejemplo 7.8. *número_bp* y *bajo* se calculan para el grafo de la figura 7.15. $BP(a)$ en la figura 7.15. Como ejemplo de la obtención de *bajo*, el recorrido en orden posterior visita *e* primero. En *e*, hay aristas de regreso (*e*, *a*) y (*e*, *b*), así que $bajo[e]$ se iguala a $\min(número_bp[e], número_bp[a], número_bp[b]) = 1$. Después se visita *d*, y $bajo[d]$ se hace igual al mínimo de $número_bp[d]$, $bajo[e]$ y $número_bp[a]$. El segundo de éstos surge porque *e* es un hijo de *d*, y el tercero, por la existencia de la arista de retroceso (*d*, *a*).



Después de obtener *bajo*, se considera cada vértice. La raíz *a* es un punto de articulación porque tiene dos hijos. El vértice *c* es un punto de articulación porque tiene un hijo *f* con $bajo[f] \geq número_bp[c]$. Los otros vértices no son puntos de articulación. □

El tiempo que consume el algoritmo anterior en un grafo de *a* aristas y $n \leq a$ vértices es $O(a)$. Es recomendable comprobar que el tiempo empleado en cada una de las tres fases puede atribuirse al vértice visitado o a una arista que parta de ese vértice, y sólo se le puede atribuir una cantidad constante de tiempo a cualquier arista o vértice en cualquier paso. Así, el tiempo total es $O(n + a)$, el cual es $O(a)$ en el supuesto de que $n \leq a$.

7.5 Pareamiento de grafos

En esta sección se bosquejará un algoritmo para resolver «problemas de pareamiento» en grafos. Un ejemplo simple de problema de pareamiento ocurre cuando se tiene un conjunto de profesores para distribuir en un conjunto de cursos. Cada profesor es competente para impartir ciertos cursos, pero no otros. Se desea asignar un curso al profesor adecuado, pero sin asignar dos profesores al mismo curso. Para ciertas distribuciones de profesores y cursos, es imposible asignar un curso a cada profesor; en tal situación, es deseable asignar tantos profesores como sea posible.

Esto se representa con un grafo como el de la figura 7.16, donde los vértices están divididos en dos conjuntos V_1 y V_2 , de modo que los vértices del conjunto V_1 representan a los profesores, y los vértices en V_2 , los cursos. Que el profesor v sea adecuado para impartir el curso w se representa con una arista (v, w) . Un grafo como éste, cuyos vértices se pueden dividir en dos grupos disjuntos y las aristas presentan un extremo en cada grupo, se conoce como *bipartido*. Asignar un profesor a un curso es equivalente a seleccionar una arista entre un vértice profesor y un vértice curso.

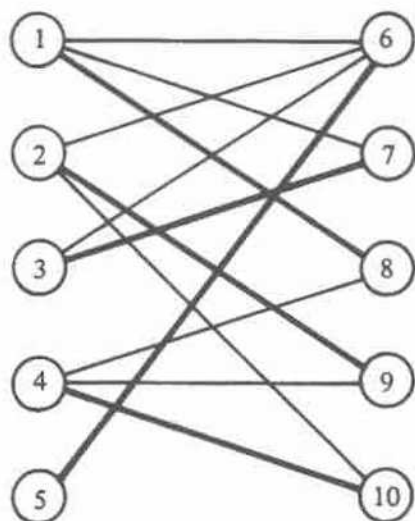


Fig. 7.16. Grafo bipartido.

El problema del pareamiento se puede formular en términos generales como sigue. Dado un grafo $G = (V, A)$, el subconjunto de las aristas de A en el que ningún par de aristas es incidente sobre el mismo vértice de V , se conoce como *pareamiento*. La tarea de la selección de subconjuntos máximos de tales aristas se denomina *problema de pareamiento maximal*, y un ejemplo son las aristas más gruesas de la figura 7.16. Un *pareamiento completo* es aquel en el que todo vértice es un punto final de alguna arista en ella. Claramente, todo pareamiento completo es maximal.

Un modo directo de encontrar pareamientos maximales, es generar en forma sistemática todos los pareamientos y luego marcar uno que tenga el mayor número de aristas. La dificultad de este método radica en que tiene un tiempo de ejecución que es una función exponencial del número de aristas.

Existen algoritmos más eficientes para la obtención de pareamientos maximales. Esos algoritmos usan de ordinario una técnica conocida como «caminos aumentados». Sea C un pareamiento en un grafo G . Un vértice v está *pareado* si es el punto final de una arista de C . Un camino que conecte dos vértices no pareados, cuyas aristas alternas estén en C , se conoce como *camino aumentado relativo a C* . Obsérvese que un camino aumentado debe tener longitud impar, y debe empezar y terminar con aristas que no estén en C . Obsérvese también que a partir de un camino aumentado A siempre es posible encontrar un pareamiento más amplio, suprimiendo de C

aquellas aristas que estén en A , y añadiendo después a C las aristas de A que no estaban inicialmente en C . Este pareamiento nuevo es $C \oplus A$, donde $\oplus$ denota «o exclusivo» en conjuntos, esto es, el nuevo pareamiento que consta de aquellas aristas que están en C o en A , pero no en ambos.

Ejemplo 7.9. La figura 7.1(a) muestra un grafo y un pareamiento C que consta de las aristas gruesas (1, 6), (3, 7) y (4, 8). El camino 2, 6, 1, 8, 4, 9 de la figura 7.17(b) es un camino aumentado relativo a C . La figura 7.18 muestra el pareamiento (1, 8), (2, 6), (3, 7), (4, 9) obtenido al eliminar aquellas aristas de C que están en el camino, y al agregar después a C las otras aristas del camino. $\square$

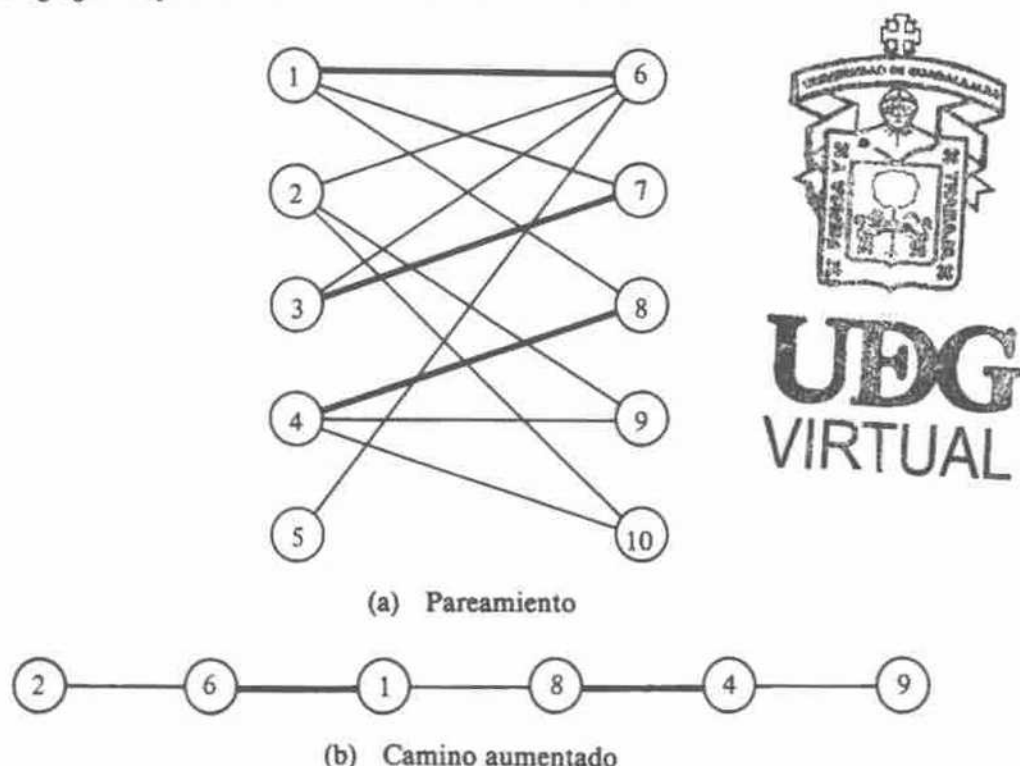


Fig. 7.17. Pareamiento y camino aumentado.

La observación clave es que C es un pareamiento maximal si, y sólo si, no existe un camino aumentado relativo a C . Esta observación es la base del algoritmo del pareamiento maximal.

Supóngase que C y D son pareamientos con $|C| < |D|$. ($|C|$ representa el número de aristas en C .) Para ver que $C \oplus D$ contiene un camino aumentado relativo a C , considérese el grafo $G' = (V, C \oplus D)$. Ya que C y D son pareamientos, cada vértice de V es un punto final de hasta una arista de C y un punto final de hasta una arista de D . Así, cada componente conexo de G' forma un camino simple (quizá un ciclo) con aristas alternando entre C y D . Cada camino que no sea un ciclo puede ser un camino aumentado relativo a C o un camino aumentado relativo a D , según

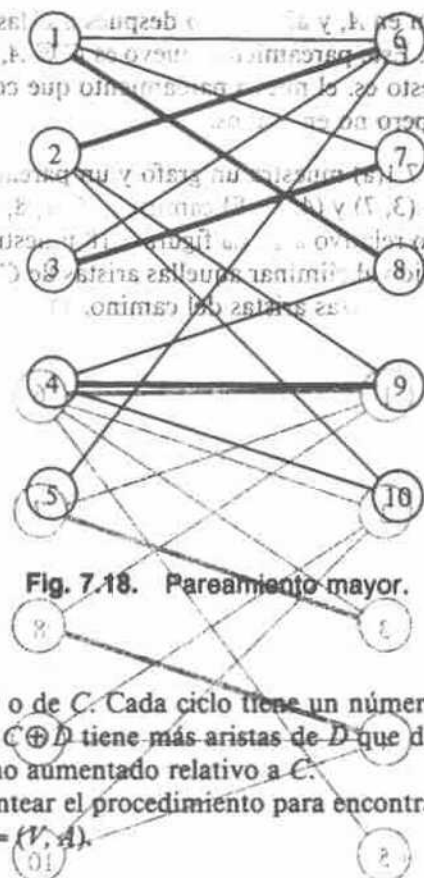


Fig. 7.18. Pareamiento mayor.

tenga más aristas de D o de C . Cada ciclo tiene un número igual de aristas de C y D . Ya que $|C| < |D|$, $C \oplus D$ tiene más aristas de D que de C , y de aquí que tenga por lo menos un camino aumentado relativo a C .

Ahora se puede plantear el procedimiento para encontrar un pareamiento maximal C para el grafo $G = (V, A)$.

1. Iniciar con $C = \emptyset$.
2. Encontrar un camino aumentado A relativo a C y reemplazar C por $C \oplus A$.
3. Repetir el paso (2) hasta que ya no existan más caminos aumentados, punto en el que C es un pareamiento maximal.

Sólo falta mostrar cómo encontrar un camino aumentado relativo a un pareamiento C , y se hará para el caso más simple, en el que G es un grafo bipartido. Se construirá un grafo de caminos aumentados para G en los niveles $i = 0, 1, 2, \dots$ por medio de un proceso similar a la búsqueda en amplitud. En el nivel $i = 0$ se inicia con algún vértice sin pareamiento. En un nivel impar i , se agregan vértices nuevos adyacentes a un vértice en un nivel $i - 1$, a través de una arista no pareada, y también se agrega esa arista. En un nivel par i , se añaden vértices nuevos adyacentes a un vértice en un nivel $i - 1$ gracias a una arista del pareamiento C , junto con esa arista. Se continúa construyendo el grafo de caminos aumentados nivel a nivel hasta que se agregue un vértice sin pareamiento en un nivel impar, o hasta que no se puedan agregar más vértices. Si un vértice sin pareamiento v se agrega en un nivel impar, el camino existente entre v y el vértice inicial del nivel cero será un camino aumentado relativo a C . Si no hay más vértices para agregar al camino aumentado del grafo, se construye otro camino aumentado empezando en un nuevo vértice inicial sin pareamiento. Si no existen más vértices nuevos sin pareamiento, entonces no ha-

brá camino aumentado relativo a C . Si hay un camino aumentado, tarde o temprano se construirá un grafo de caminos aumentados que termine en un vértice sin pareamiento en un nivel impar.

Ejemplo 7.10. La figura 7.19 ilustra el grafo de caminos aumentados para la figura 7.17(a) relativo al pareamiento de la figura 7.18, en la cual se ha escogido 5 como el vértice sin pareamiento en el nivel 0. En el nivel 1 se agrega la arista sin pareamiento (5, 6). En el nivel 2 se agrega la arista de pareamiento (6, 2). En el nivel 3 se puede agregar cualquiera de las aristas sin pareamiento (2, 9) o (2, 10). Dado que los vértices 9 y 10 están actualmente sin pareamiento, se puede terminar la construcción del grafo de caminos aumentados después de incorporar cualquiera de esos vértices. Los caminos 9, 2, 6, 5 y 10, 2, 6, 5 son caminos aumentados relativos al pareamiento de la figura 7.18. $\square$

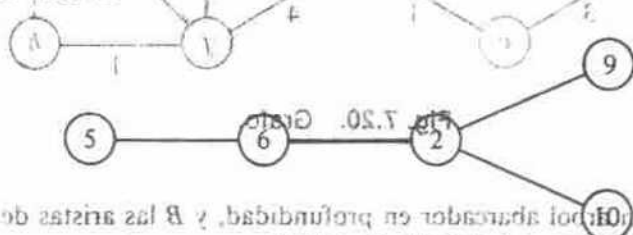


Fig. 7.19 Grafo de caminos aumentados.

Supóngase que G tiene n vértices y a aristas. La construcción de los grafos de caminos aumentados para un pareamiento dado, lleva un tiempo $O(a)$ si se representan las aristas por medio de una lista de adyacencia. Así, encontrar cada camino aumentado nuevo lleva un tiempo $O(a)$. Para encontrar un pareamiento maximal, se construyen hasta $n/2$ caminos aumentados, ya que cada uno aumenta por lo menos en una arista el pareamiento actual. Por tanto, puede encontrarse un pareamiento maximal en un tiempo $O(na)$ para un grafo bipartido G .

Ejercicios

- 7.1 Describese un algoritmo para insertar y eliminar aristas en un grafo no dirigido representado por medio de una lista de adyacencia. Recuerdese que una arista (i, j) aparece en la lista de adyacencia de los vértices i y j .
- 7.2 Modifíquese la representación con lista de adyacencia de un grafo no dirigido, para que la primera arista de la lista de un vértice se pueda eliminar en un tiempo constante. Escribese un algoritmo para eliminar la primera arista en un vértice utilizando esta representación. *Sugerencia:* ¿Cómo se puede hacer que las dos celdas que representan la arista (i, j) se encuentren una a la otra con rapidez?

7.3 Considérese el grafo de la figura 7.20. Encuéntrese,

- Un árbol abarcador de costo mínimo con el algoritmo de Prim.
- Un árbol abarcador de costo mínimo con el algoritmo de Kruskal.
- Un árbol abarcador en profundidad empezando en a y en d .
- Un árbol abarcador en amplitud empezando en a y en d .

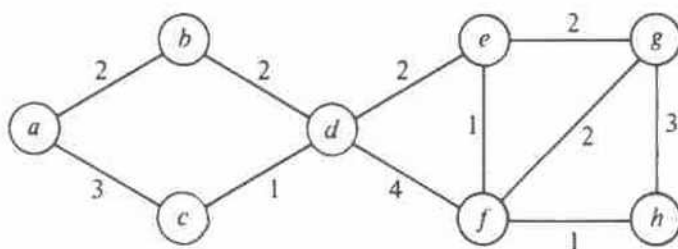


Fig. 7.20. Grafo.

7.4 Sea T un árbol abarcador en profundidad, y B las aristas de retroceso de un grafo conexo no dirigido $G = (V, A)$.

- Demuéstrese que cuando cada arista de retroceso de B se agrega a T , se obtiene un ciclo único. Llámese tal ciclo un ciclo *básico*.
 - La combinación lineal de ciclos $C_1, C_2, \dots, C_n$ es $C_1 \oplus C_2 \oplus \dots \oplus C_n$. Pruébese que se obtiene un ciclo de la combinación lineal de dos ciclos no disjuntos distintos.
 - Demuéstrese que todo ciclo en G puede expresarse como una combinación lineal de ciclos básicos.
- *7.5 Sea $G = (V, A)$ un grafo, y R una relación en V tal que $u R v$ si, y sólo si, u y v están dentro de un ciclo común (no necesariamente simple). Pruébese que R es una relación de equivalencia en V .
- 7.6 Implántense los algoritmos de Prim y de Kruskal. Compárense los tiempos de ejecución de sus programas en una colección de grafos «aleatorios».
- 7.7 Escribbase un programa para encontrar todos los componentes conexos de un grafo.
- 7.8 Escribbase un programa de orden $O(n)$ para determinar si un grafo de n vértices contiene algún ciclo.
- 7.9 Escribbase un programa para enumerar todos los ciclos simples de un grafo. ¿Cuántos ciclos de este tipo puede haber? ¿Cuál es la complejidad en tiempo del programa?
- 7.10 Muéstrese que todas las aristas de una búsqueda en amplitud son de árbol o cruzadas.

- 7.11 Implántese el algoritmo para encontrar los puntos de articulación, analizado en la sección 7.4.
- *7.12 Sea $G = (V, A)$ un grafo *completo*, esto es, un grafo en el cual existe una arista entre todo par de vértices distintos. Sea $G' = (V, A')$ un grafo dirigido en el cual A' es A que da a cada arista una orientación arbitraria. Muéstrase que G' tiene un camino dirigido que incluye todos los vértices exactamente una vez.
- **7.13 Muéstrase que un grafo completo de n vértices tiene n^{n-2} árboles abarcadores.
- 7.14 Encuéntrense todos los pareamientos maximales para la figura 7.16.
- 7.15 Escribese un programa para encontrar un pareamiento maximal para un grafo bipartido.
- 7.16 Sea C un pareamiento y c el número de aristas en un pareamiento maximal.
- Pruébese que existe un camino aumentado relativo a C cuya longitud es $2(|C|/c - |C|) + 1$ como máximo.
 - Pruébese que si A es el camino aumentado más corto relativo a C y si A' es un camino aumentado relativo a $C \oplus A$, entonces $|A'| \geq |A| + |A \cap A'|$.
- *7.17 Pruébese que un grafo es bipartido si, y sólo si, no tiene ciclos de longitud impar. Dése un ejemplo de un grafo no bipartido para el cual la técnica del grafo de caminos aumentados no funcione.
- 7.18 Sean C y D los pareamientos de un grafo bipartido. Pruébese que $C \oplus D$ tiene al menos $|C| - |D|$ vértices disjuntos en los caminos aumentados relativos a C .

Notas bibliográficas

Los métodos para construcción de árboles abarcadores minimales se han estudiado por lo menos desde Boruvka [1926]. Los dos algoritmos dados en este capítulo están basados en Kruskal [1956] y Prim [1957]. Johnson [1975] muestra cómo los árboles k -arios parcialmente ordenados se pueden usar para realizar el algoritmo de Prim. Cheriton y Tarjan [1978] y Yao [1975] presentan algoritmos $O(\log \log n)$ para árboles abarcadores. Tarjan [1981] presenta una historia y perspectiva completas de los algoritmos para árboles abarcadores.

Hopcroft y Tarjan [1973] y Tarjan [1972] popularizaron el uso de la búsqueda en profundidad en algoritmos para grafos. De ahí procede el algoritmo para componentes biconexos.

El pareamiento de grafos lo estudió Hall [1948], y los caminos aumentados, Berge [1957] y Edmonds [1965]. Hopcroft y Karp [1973] dan un algoritmo $O(n^{2.5})$ para pareamientos maximales en grafos bipartidos, y Micali y Vazirani [1980] dan un algoritmo $O(\sqrt{|V|} \cdot |A|)$ para pareamiento maximal en grafos generales. En Papadimitriou y Steiglitz [1982] hay un buen análisis del pareamiento general.

Clasificación

El proceso de clasificación u ordenamiento de una lista de objetos de acuerdo con algún orden lineal, como $\leq$ para números, es tan fundamental, y se realiza con tanta frecuencia, que justifica una revisión cuidadosa del tema. La clasificación se divide en dos partes: interna y externa. La clasificación interna se produce en la memoria principal del computador, donde es posible aprovechar la capacidad del acceso aleatorio en distintas formas. La clasificación externa es necesaria cuando el número de objetos a ordenar es demasiado grande para caber en la memoria principal. En ese caso, el «embotellamiento» suele ser la transferencia de datos entre el almacenamiento principal y el secundario, y es imprescindible mover grandes bloques de datos para lograr mayor eficiencia. El hecho de que sea más conveniente mover en un solo bloque los datos físicamente contiguos restringe las clases de algoritmos de clasificación externa posibles. La clasificación externa se cubrirá en el capítulo 11.

8.1 El modelo de clasificación interna

En este capítulo, se presentarán los principales algoritmos de clasificación interna. Los algoritmos más simples de ordinario requieren un tiempo $O(n^2)$ para clasificar n objetos y sólo son útiles para listas pequeñas. Uno de los algoritmos de clasificación más populares es la clasificación rápida (*quicksort*), que lleva en promedio un tiempo $O(n \log n)$. La clasificación rápida funciona muy bien para la mayor parte de las aplicaciones comunes, sin embargo, en el peor caso lleva un tiempo $O(n^2)$. Existen otros métodos, como la clasificación por montículos (*heapsort*) y la clasificación por intercalación (*mergesort*), que llevan un tiempo $O(n \log n)$ en el peor caso, aunque su comportamiento en el caso promedio quizá no sea tan bueno como el de la clasificación rápida. La clasificación por intercalación, sin embargo, es una buena elección para clasificación externa. Se considerarán otros algoritmos llamados «clasificación por cubetas» o «clasificación por urnas». Estos algoritmos operan sólo con clases especiales de datos, como los enteros escogidos de un intervalo limitado, pero cuando son aplicables son muy rápidos, pues requieren sólo un tiempo $O(n)$ en el peor caso.

En este capítulo se supone que los objetos a clasificar son registros que contienen uno o más campos. Uno de los campos, llamado *clave*, es de un tipo para el cual está definida la relación de ordenamiento lineal $\leq$. Enteros, reales, y arreglos de caracteres son ejemplos comunes de estos tipos, aunque por lo general se puede utili-

zar cualquier tipo de clave para la cual estén definidas las relaciones «menor o igual que» o «menor que».

El problema de la *clasificación* consiste en ordenar una secuencia de registros de tal forma que los valores de sus claves formen una secuencia no decreciente. Esto es, dados los registros $r_1, r_2, \dots, r_n$ con valores de clave $k_1, k_2, \dots, k_n$ respectivamente, debe resultar la misma secuencia de registros en orden $r_{i_1}, r_{i_2}, \dots, r_{i_n}$ tal que $k_{i_1} \leq k_{i_2} \leq \dots \leq k_{i_n}$. No es necesario que todos los registros tengan valores distintos, ni que los registros con claves iguales aparezcan en un orden particular.

Se emplearán varios criterios para evaluar el tiempo de ejecución de un algoritmo de clasificación interna. La primera medición, y también la más común, es el número de pasos requeridos por el algoritmo para clasificar n registros. Otra medición frecuente es el número de comparaciones entre claves que debe efectuarse para clasificar n registros; esto resulta muy útil cuando la comparación entre un par de claves es una operación relativamente costosa, como sucede cuando las claves son grandes cadenas de caracteres. Si el tamaño del registro es grande, puede ser también conveniente contar las veces que debe moverse. La aplicación manual hace evidente la medida del costo apropiada.

8.2 Algoritmos esquemas simples de clasificación

Tal vez uno de los métodos de clasificación más simples que pueda haber es un algoritmo llamado «clasificación de burbuja» (*bubblesort*). La idea básica de la clasificación de burbuja es imaginar que los registros a ordenar están almacenados en un arreglo vertical. Los registros con claves menores son «ligeros» y suben. Se recorre varias veces el arreglo de abajo hacia arriba, y al hacer esto, si hay dos elementos adyacentes que no están en orden, esto es, si el «más ligero» está abajo, se invierten. El efecto producido por esta operación es que en el primer recorrido, el registro «más ligero» de todos, es decir, el registro que posee la clave con menor valor, sube hasta la superficie (o parte superior del arreglo). En el segundo recorrido, la segunda clave menor sube hasta la segunda posición, y así sucesivamente. En este recorrido no es necesario subir hasta la posición uno, porque ya se sabe que la clave menor se encuentra ahí. En general, el recorrido i no intenta pasar más allá de la posición i . Se presenta el esbozo del algoritmo en la figura 8.1, con el supuesto de que el arreglo A es un `array[1..n]` of tipo_registro, y n es el número de registros. Aquí y en el resto del capítulo se supondrá que un campo llamado *clave* contiene el valor de la clave de cada registro.

```

(1) for i := 1 to n-1 do
(2)   for j := n downto i+1 do
(3)     if A[j].clave < A[j-1].clave then
(4)       intercambia(A[j], A[j-1])

```

Fig. 8.1. Algoritmo de clasificación de burbuja.

El procedimiento *intercambia* se utiliza en varios algoritmos de clasificación y se define en la figura 8.2.

```

procedure intercambia ( var x, y: tipo_registro )
  { intercambia cambia los valores de x e y }
  var
    temp: tipo_registro;
  begin
    temp := x;
    x := y;
    y := temp
  end; { intercambia }

```

Fig. 8.2. El procedimiento *intercambia*.

Ejemplo 8.1. La figura 8.3 muestra una lista de volcanes famosos, y el año en que hicieron erupción.

NOMBRE	AÑO
Peléé	1902
Etna	1669
Krakatoa	1883
Agung	1963
Vesubio	79
Santa Elena	1980

Fig. 8.3. Volcanes famosos.

Para este ejemplo, se emplean las siguientes definiciones de tipos:

```

type
  tipo_clave = array[1..10] of char;
  tipo_registro = record
    clave: tipo_clave; {nombre del volcán}
    año: integer
  end;

```

El algoritmo de clasificación de burbuja de la figura 8.1, aplicado a la lista de la figura 8.3, clasifica la lista en orden alfabético de nombres, si la relación $\leq$ en objetos con este tipo de claves es el orden lexicográfico habitual. En la figura 8.4, se muestran los cinco pasos dados por el algoritmo cuando $n=6$. Las líneas indican el punto sobre el cual se sabe que los nombres son los más pequeños (en orden alfabético) y ocupan el lugar correcto. Sin embargo, después de $i=5$, cuando todos excepto el último registro se han colocado en su lugar, el último también debe estar en el lugar correcto, y el algoritmo termina.

Al principio del primer recorrido, el Santa Elena sobrepasa a Vesubio, pero no a Agung. En el resto de este recorrido, Agung sube hasta la parte superior. En el se-

gundo recorrido, Etna sube a la posición 2. En el tercer recorrido, Krakatoa sobrepasa a Pelée, y la lista queda en orden lexicográfico; sin embargo, de acuerdo con el algoritmo de la figura 8.1, se hacen dos recorridos adicionales. $\square$

Pelée	<u>Agung</u>	Agung	Agung	Agung	Agung
Etna	Pelée	<u>Etna</u>	Etna	Etna	Etna
Krakatoa	Etna	Pelée	<u>Krakatoa</u>	Krakatoa	Krakatoa
Agung	Krakatoa	Krakatoa	Pelée	<u>Pelée</u>	Pelée
Vesubio	Santa Elena	Santa Elena	Santa Elena	Santa Elena	<u>Santa Elena</u>
Santa Elena	Vesubio	Vesubio	Vesubio	Vesubio	<u>Vesubio</u>
inicial	después de $i=1$	después de $i=2$	después de $i=3$	después de $i=4$	después de $i=5$

Fig. 8.4. Recorridos de la clasificación de burbuja.

Clasificación por inserción

El segundo método de clasificación a considerar se denomina «clasificación por inserción», porque en el i -ésimo recorrido se «inserta» el i -ésimo elemento $A[i]$ en el lugar correcto, entre $A[1]$, $A[2]$, ..., $A[i-1]$, los cuales fueron ordenados previamente. Después de hacer esta inserción, se encuentran clasificados los registros colocados en $A[1]$, ..., $A[i]$. Esto es, se ejecuta

```

for  $i := 2$  to  $n$  do
  mover  $A[i]$  hacia la posición  $j \leq i$  tal que
     $A[i] < A[j]$  para  $j \leq k < i$ , y
     $A[i] \geq A[j-1]$  o  $j = 1$ 

```

Para facilitar el proceso de mover $A[i]$, es útil introducir un elemento $A[0]$, cuya clave tiene un valor menor que el de cualquier clave existente en $A[1]$, ..., $A[n]$. Se postulará la existencia de una constante $-\infty$ de tipo tipo_clave que es menor que la clave de cualquier registro que pueda aparecer en la práctica. Si ninguna constante $-\infty$ se puede utilizar con seguridad, se debe probar primero si $j = 1$ al decidir la inserción de $A[i]$ antes de la posición j , y si no, comparar $A[i]$ (que se encuentra ahora en la posición j) con $A[j-1]$. El programa completo se muestra en la figura 8.5.

```

(1)  $A[0].clave := -\infty$ ;
(2) for  $i := 2$  to  $n$  do begin
(3)    $j := i$ ;
(4)   while  $A[j] < A[j-1]$  do begin
(5)     intercambia( $A[j]$ ,  $A[j-1]$ );
(6)      $j := j-1$ 
  end
end

```

Fig. 8.5. Clasificación por inserción.

Ejemplo 8.2. En la figura 8.6 se muestra la lista inicial de la figura 8.3 y el resultado de los recorridos sucesivos de la clasificación por inserción para $i = 2, 3, 4, 5, 6$. Después de cada recorrido está garantizado que los elementos por arriba de la línea estarán ordenados entre sí, aunque su orden no tenga relación con los registros encontrados bajo la línea, los cuales se insertarán después.

inicial	después de $i=2$	después de $i=3$	después de $i=4$	después de $i=5$	después de $i=6$
Etna Pelée Etna Krakatoa Agung Vesubio Santa Elena	Etna Pelée Pelée Krakatoa Agung Vesubio Santa Elena	Etna Pelée Krakatoa Pelée Agung Vesubio Santa Elena	Agung Etna Krakatoa Pelée Vesubio Santa Elena	Agung Etna Krakatoa Pelée Vesubio Santa Elena	Agung Etna Vesubio Krakatoa Pelée Santa Elena

Fig. 8.6. Recorridos de la clasificación por inserción.

Clasificación por selección. El segundo método de clasificación se denomina «clasificación por selección», porque en el i -ésimo recorrido se «selecciona» el elemento más pequeño entre $A[i], A[i+1], \dots, A[n]$, los cuales están ordenados previamente. Después de hacer esta selección se encuentra la posición en la lista en la que debe ser colocado. La idea en que se sustenta la «clasificación por selección» también es elemental. En el i -ésimo recorrido se selecciona el registro con la clave más pequeña, entre $A[i], \dots, A[n]$, y se intercambia con $A[i]$. Como resultado, después de i pasadas, los i registros menores ocuparán $A[1], \dots, A[i]$, en el orden clasificado. Esto es, la clasificación por selección puede describirse por

for $i := 1$ to $n-1$ do

seleccionar el más pequeño entre $A[i], \dots, A[n]$ e
intercambiarlo con $A[i]$;

Un programa más completo se muestra en la figura 8.7.
Ejemplo 8.3. En la figura 8.8 se muestran los recorridos de la clasificación por selección de la lista de la figura 8.3. Por ejemplo, en el recorrido 1, el último valor de índice_menor es 4, la posición de Agung, que se intercambia con Vesubio en $A[1]$.

Las líneas de la figura 8.8 indican el punto sobre el cual se sabe que los elementos menores aparecen clasificados. Después de $n-1$ recorridos, el registro $A[n]$, Vesubio en la figura 8.8, está también en el lugar correcto, ya que es el elemento que se sabe que no está entre los $n-1$ más pequeños.

Complejidad de tiempo de los métodos

Las clasificaciones de burbuja, por inserción y por selección llevan un tiempo $O(n^2)$, y llevarán un tiempo $O(n^2)$ en buena parte de las secuencias de entrada de n elemen-

var *clave_menor*, *tipo_clave*; *La clave menor encontrada actualmente en un*
recorrido a través de A[i], ..., A[n].
índice_menor, integer, *la posición de clave_menor;*
 begin
 (1) for *i* := 1 to *n* do begin
 (2) *índice_menor* := *i*;
 (3) *clave_menor* := *A[i].clave*;
 (4) for *j* := *i + 1* to *n* do
 (5) if *A[j].clave* < *clave_menor* then begin
 (6) *clave_menor* := *A[j].clave*;
 (7) *índice_menor* := *j*;
 end;
 (8) *intercambia*(*A[i]*, *A[índice_menor]*);
 end;

Fig. 8.7. Clasificación por selección.

	<u>Agung</u>	<u>Agung</u>	<u>Agung</u>	<u>Agung</u>	<u>Agung</u>
Peléé	<u>Etna</u>	<u>Etna</u>	<u>Etna</u>	<u>Etna</u>	<u>Etna</u>
Krakatoa	Krakatoa	Krakatoa	Krakatoa	Krakatoa	Krakatoa
Agung	Peléé	Peléé	Peléé	Peléé	Peléé
Vesubio	Vesubio	Vesubio	Vesubio	Vesubio	Santa Elena
Santa Elena	Santa Elena	Santa Elena	Santa Elena	Santa Elena	<u>Vesubio</u>
inicial	después	después	después	después	después
	de <i>i</i> = 1	de <i>i</i> = 2	de <i>i</i> = 3	de <i>i</i> = 4	de <i>i</i> = 5

Fig. 8.8. Recorridos de la clasificación por selección.

tos: Considérese la clasificación de burbuja de la figura 8.1. Sin importar qué tipo de registro sea, *intercambia* lleva un tiempo constante. Así, las líneas (3) y (4) de la figura 8.1 consumen hasta c_1 unidades de tiempo para alguna constante c_1 . Por tanto, para un valor fijo de i , el ciclo de las líneas (2) a (4) lleva hasta $c_1(n-1)$ pasos, para alguna constante c_1 ; la última constante es algo mayor que c_1 para justificar los decrementos y pruebas de j . En consecuencia, el programa completo requiere

$$c_3n + \sum_{i=1}^{n-1} c_2(n-i) = \frac{1}{2}c_2n^2 + (c_3 - \frac{1}{2}c_2)n$$
 pasos, donde el término c_3n tiene en cuenta los incrementos y pruebas de i . Como la última fórmula no excede $(c_2/2 + c_3)n^2$, para $n \geq 1$, se observa que la complejidad de tiempo de la clasificación de burbuja es $O(n^2)$. El algoritmo requiere $\Omega(n^2)$ pasos,

ya que aunque no se necesiten intercambios (es decir, si la entrada ya estuviera clasificada), la prueba de la línea (3) se ejecutaría $n(n-1)/2$ veces.

A continuación, considérese la clasificación por inserción de la figura 8.5. El ciclo **while** de las líneas (4) a (6) de la figura 8.5 no puede llevar más de $O(i)$ pasos, ya que j toma un valor inicial i en la línea (3) y decrece en cada ciclo. El ciclo termina cuando $j = 1$, ya que $A[0]$ es $-\infty$, lo cual hace que la prueba de la línea (4) sea falsa cuando $j = 1$. Se puede concluir que el ciclo **for** de las líneas (2) a (6) lleva hasta $c \sum_{i=2}^n i$ pasos para alguna constante c . Esta suma es $O(n^2)$.

Sería aconsejable comprobar que si en un inicio el arreglo está clasificado en orden inverso, se ejecutará $i-1$ veces el ciclo **while** de las líneas (4) a (6), así que la línea (4) se ejecuta $\sum_{i=2}^n (i-1) = n(n-1)/2$ veces. Por tanto, la clasificación por inserción requiere un tiempo $\Omega(n^2)$ en el peor caso. Se puede demostrar que este límite menor interno vale también para el caso promedio.

Por último, considérese la clasificación por selección de la figura 8.7. Se puede comprobar que el ciclo **for** interno de las líneas (4) a (7) lleva un tiempo $O(n-i)$, puesto que j va desde $i+1$ hasta n . Así, el tiempo total que requiere el algoritmo es $c \sum_{i=1}^{n-1} (n-i)$, para alguna constante c . Esta suma, que es $cn(n-1)/2$, se observa que es $O(n^2)$. Por el contrario, se puede mostrar que al menos la línea (5) se ejecuta $\sum_{i=1}^{n-1} \sum_{j=i+1}^n (1) = n(n-1)/2$ veces, sin importar el valor inicial del arreglo A , así que la clasificación por selección lleva un tiempo $\Omega(n^2)$, tanto en el peor caso, como en el caso promedio.

Cuenta de intercambios

Si el tamaño de los registros es grande, el procedimiento *intercambia*, único lugar en los tres algoritmos donde se copian registros, llevará más tiempo que cualquier otro paso, como la comparación de claves o los cálculos en los índices del arreglo. Así, mientras que los tres algoritmos llevan un tiempo proporcional a n^2 , se pueden comparar con más detalle al contar las veces que se usa *intercambia*.

Para comenzar, la clasificación de burbuja ejecuta el paso de intercambio de la línea (4) en la figura 8.1

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n (1) = n(n-1)/2$$

veces a lo sumo, o unas $n^2/2$ veces. Pero dado que la ejecución de la línea (4) depende del resultado obtenido en la prueba de la línea (3), cabe esperar que el número real de intercambios sea mucho menor que $n^2/2$.

De hecho, la clasificación de burbuja, en promedio, intercambia elementos exactamente la mitad de veces, así que el número de intercambios esperado, si todas las

secuencias iniciales tienen igual probabilidad, será aproximadamente $n^2/4$. Para corroborar esto, considérense dos listas iniciales de claves mutuamente inversas: $L_1 = k_1, k_2, \dots, k_n$ y $L_2 = k_n, k_{n-1}, \dots, k_1$. Un intercambio es la única forma de que k_i y k_j puedan cruzarse si están inicialmente fuera de orden; pero, k_i y k_j lo están sólo en una de las listas L_1 o L_2 . Así, el número total de intercambios ejecutados cuando se aplica la clasificación de burbuja a L_1 y L_2 es igual al número de pares de elementos, esto es, $\binom{n}{2}$ o $n(n-1)/2$. Por tanto, el número de intercambios promedio para L_1 y L_2 es $n(n-1)/4$ o unas $n^2/4$. Ya que todas las clasificaciones posibles pueden aparearse con sus inversas, como sucedió con L_1 y L_2 , el número promedio de intercambios en todas las clasificaciones también será aproximadamente $n^2/4$.

El número de intercambios efectuados en la clasificación por inserción es, en promedio, idéntico al de la clasificación de burbuja. Aplicando el mismo argumento, cada par de elementos se intercambiará en una lista L o en su inversa, pero nunca en ambas.

Sin embargo, en caso de que *intercambia* sea una operación costosa, es fácil observar que la clasificación por selección es superior a las clasificaciones de burbuja y por inserción. La línea (8) de la figura 8.7 se encuentra fuera del ciclo interno del algoritmo de clasificación por selección, por lo que se ejecuta exactamente $n-1$ veces en un arreglo de longitud n . Como la línea (8) contiene la única llamada a *intercambia*, la velocidad de crecimiento en el número de intercambios en la clasificación por selección, que es $O(n)$, es menor que las tasas de crecimiento del número de intercambios de los otros dos algoritmos, que es $O(n^2)$. A diferencia de las clasificaciones de burbuja y por inserción, la clasificación por selección permite a los elementos «saltar» sobre grandes cantidades de elementos sin necesidad de intercambiarlos entre sí individualmente.

Cuando los registros sean grandes y los intercambios costosos, una estrategia muy útil es mantener un arreglo de apuntadores a registros por medio de cualquier algoritmo. Entonces se pueden intercambiar apuntadores en lugar de registros. Una vez que los apuntadores a registros se han dispuesto en el orden apropiado, los registros pueden disponerse en el orden clasificado final en un tiempo $O(n)$.

Limitaciones de los algoritmos simples

Se debe tener presente que los algoritmos mencionados en esta sección tienen un tiempo de ejecución $O(n^2)$, tanto en el peor caso como en el promedio. Así, para una n grande, ninguno de esos algoritmos se compara de modo favorable con los algoritmos $O(n \log n)$ que se analizarán en las siguientes secciones. El valor de n para el cual esos algoritmos más complejos se hacen mejores que los simples, depende de diversos factores, como la calidad del código objeto generado por el compilador, la máquina con que se ejecutan los programas y el tamaño de los registros que se deben intercambiar. La experimentación con un programa que registre tiempos de ejecución (*profiler*) es una buena forma de determinar el punto de ruptura. Una regla razonable es que a menos que n sea aproximadamente 100, puede ser una pérdida de tiempo implantar un algoritmo más complicado que los estudiados en esta sección. La clasificación de Shell, una generalización de la clasificación de burbuja, es un algo-

ritmo de clasificación $O(n^{1.5})$ simple, muy sencillo de ampliar y razonablemente eficiente para valores modestos de n . La clasificación de Shell se presenta en el ejercicio 8.3.

8.3 Clasificación rápida (*quicksort*)

El primer algoritmo $O(n \log n)$ que se estudia †, y tal vez el más eficiente para clasificación interna, recibe el nombre de «clasificación rápida» (*quicksort*). La esencia del método consiste en clasificar un arreglo $A[1], \dots, A[n]$ tomando en el arreglo un valor clave v como elemento *pivote*, alrededor del cual reorganizar los elementos del arreglo. Es de esperar que el pivote esté cercano al valor medio de la clave del arreglo, de forma que esté precedido por una mitad de las claves y seguido por la otra mitad. Se permutan los elementos del arreglo con el fin de que para alguna j , todos los registros con claves menores que v aparezcan en $A[1], \dots, A[j]$, y todos aquellos con claves v o mayores aparezcan en $A[j+1], \dots, A[n]$. Después, se aplica recursivamente la clasificación rápida a $A[1], \dots, A[j]$ y a $A[j+1], \dots, A[n]$ para clasificar ambos grupos de elementos. Dado que todas las claves del primer grupo preceden a todas las claves del segundo grupo, todo el arreglo quedará ordenado.

Ejemplo 8.4. En la figura 8.9 se muestran los pasos recursivos que la clasificación rápida puede realizar para clasificar la secuencia de enteros 3, 1, 4, 1, 5, 9, 2, 6, 5, 3. En cada caso, se ha escogido como valor v al mayor de los dos valores distintos que se encuentran más a la izquierda. La recursión termina al descubrir que la porción del arreglo que se debe clasificar consta de claves idénticas. Se ha mostrado que cada nivel consta de dos pasos, uno antes de dividir cada subarreglo, y el segundo, después. La reorganización de los registros que tiene lugar durante la división se explicará en seguida. □

Ahora se inicia el diseño del procedimiento recursivo *quicksort*(i, j) que opera en un arreglo A con elementos $A[1], \dots, A[n]$, definido de manera externa al procedimiento. *quicksort*(i, j) ordena desde $A[i]$ hasta $A[j]$ en el mismo lugar. En la figura 8.10 se muestra un esbozo del procedimiento. Obsérvese que si todos los elementos $A[i], \dots, A[j]$ tienen la misma clave, el procedimiento no afecta a A .

Se empieza desarrollando una función *encuentra_pivote* que obtiene la prueba de la línea (1) de la figura 8.10, para determinar si todas las claves $A[i], \dots, A[j]$ tienen el mismo valor. Si *encuentra_pivote* no encuentra dos claves distintas, devuelve 0. De otro modo, devuelve el índice de la mayor de las dos primeras claves diferentes, la cual se convierte en el elemento pivote. La función *encuentra_pivote* está escrita en la figura 8.11.

Luego, se aplica la línea (3) de la figura 8.10, donde se enfrenta el problema de la permutación de $A[i], \dots, A[j]$, en el mismo lugar ††, de manera que todas las cla-

† Técnicamente, la clasificación rápida es $O(n \log n)$ sólo en el caso promedio; en el peor caso es $O(n^2)$.

†† Podrían copiarse $A[i], \dots, A[j]$ y ordenarlos conforme se toman, para copiar el resultado otra vez en $A[i], \dots, A[j]$. No se hace así porque se malgasta espacio y llevará más tiempo que el método utilizado aquí.

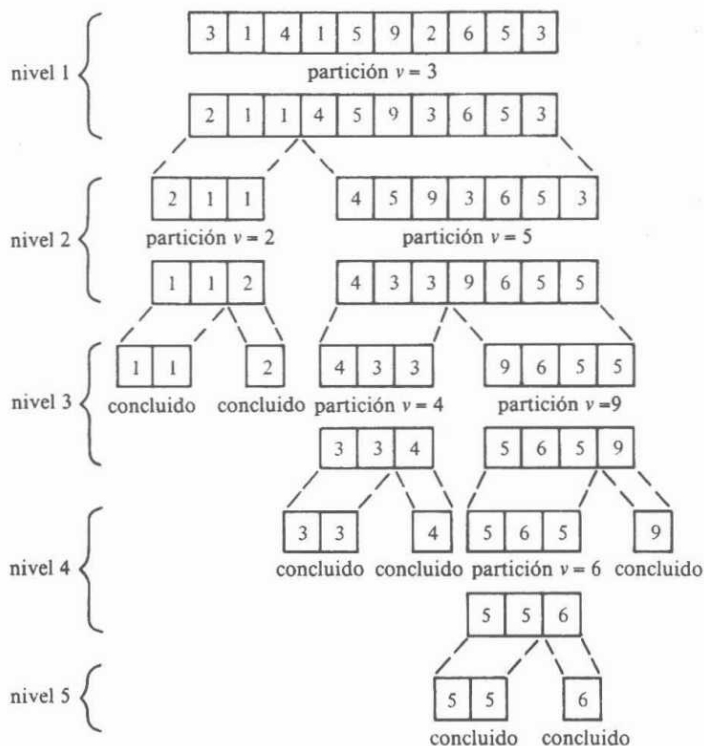


Fig. 8.9. Operación de la clasificación rápida.

ves menores que el valor del pivote aparecen a la izquierda de las demás. Para realizar esto, se introducen dos cursores, z y d , en un principio, en los extremos izquierdo y derecho, respectivamente, de la porción de A que se está clasificando. Los elementos a la izquierda de z , esto es, $A[i], \dots, A[z-1]$, siempre tendrán claves menores que el pivote. Los elementos a la derecha de d , esto es, $A[d+1], \dots, A[j]$, tendrán claves mayores o iguales al pivote, y los elementos del centro estarán mezclados, como se sugiere en la figura 8.12.

En un comienzo, $i = z$ y $j = d$ para que la proposición anterior se siga cumpliendo, ya que no existe nada a la izquierda de z ni a la derecha de d . Se efectúan varias veces los pasos siguientes, los cuales mueven z a la derecha y d a la izquierda, hasta que terminen cruzándose, momento en el que $A[i], \dots, A[z-1]$ tendrán todas las claves menores que el pivote y $A[d+1], \dots, A[j]$ todas las claves mayores o iguales que el pivote.

- (1) if de $A[i]$ a $A[j]$ existen al menos dos claves distintas then begin
 - (2) sea v la mayor de las dos claves distintas encontradas;
 - (3) permutar $A[i], \dots, A[j]$ de manera que para alguna k entre $i+1$ y j , $A[i], \dots, A[k-1]$ tengan claves menores que v y los elementos $A[k], \dots, A[j]$ tengan claves $\geq v$
 - (4) quicksort($i, k-1$);
 - (5) quicksort(k, j)
- end

Fig. 8.10. Esbozo de la clasificación rápida.

```
function encuentra_pivote ( i, j: integer ): integer;
{ devuelve 0 si  $A[i], \dots, A[j]$  tienen claves idénticas; de otra forma, devuelve el índice de la mayor de las dos claves diferentes de más a la izquierda }
var
  primera_clave: tipo_clave; { valor de la primera clave encontrada, es decir,  $A[i].clave$  }
  k: integer; { va de izquierda a derecha buscando una clave diferente }
begin
  primera_clave :=  $A[i].clave$ ;
  for k := i + 1 to j do { rastrea en busca de una clave distinta }
    if  $A[k].clave > primera\_clave$  then { selecciona la clave mayor }
      return (k)
    else if  $A[k].clave < primera\_clave$  then
      return (i);
  return (0) { nunca se encontraron dos claves diferentes }
end; { encuentra_pivote }
```

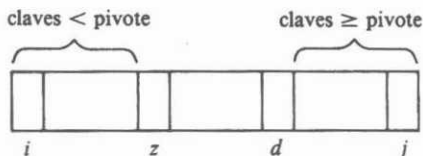
Fig. 8.11. Procedimiento *encuentra_pivote*.

Fig. 8.12. Situación durante el proceso de permutación.

1. *Rastrear*. Mueve z a la derecha en los registros cuyas claves sean menores que el pivote. Mueve d a la izquierda en las claves mayores o iguales que pivote. Obsérvese que esta selección del pivote por *encuentra_pivote* garantiza que por lo menos existe una clave menor y una no menor que el pivote, de modo que z y d con seguridad se detendrán antes de quedar fuera del intervalo de i a j .

2. *Probar*. Si $z > d$ (lo que en la práctica significa que $z = d + 1$), entonces se ha dividido $A[i], \dots, A[j]$ en forma satisfactoria, lo cual basta.
3. *Desviar*. Si $z < d$ (obsérvese que no se puede parar durante el rastreo con $z = d$, porque uno u otro se moverá más allá de una clave dada), se intercambia $A[z]$ con $A[d]$. Después de hacerlo, $A[z]$ tiene una clave menor que el pivote y $A[d]$ tiene una clave por lo menos igual que el pivote, y se sabrá que en la siguiente fase de rastreo z se moverá al menos una posición a la derecha, en la $A[d]$ anterior y d se moverá al menos una posición a la izquierda.

El ciclo anterior es poco apropiado, ya que la prueba que lo termina está justo en la mitad. Para ponerla en la forma de un ciclo **repeat**, se pasa la fase *desviar* al principio. El efecto es que inicialmente, cuando $i = z$ y $j = d$, se intercambia $A[i]$ con $A[j]$. Esto puede ser o no correcto, pero no es muy importante, pues se supone que en un principio no existe ningún orden en particular para las claves entre $A[i], \dots, A[j]$. Sin embargo, es necesario comprender este «truco» y no dejarse inquietar por él. La función *partición*, que realiza las operaciones anteriores y devuelve z , el punto en el cual empieza la mitad superior del arreglo dividido, se muestra en la figura 8.13.

En este momento ya es posible esbozar el programa para la clasificación rápida de la figura 8.10; el programa final se muestra en la figura 8.14. Para clasificar un arreglo A de tipo `array[1..n]` of tipo_registro sólo se llama a `quicksort(1, n)`.

```

function partición (  $i, j$ : integer; pivote: tipo_clave ): integer;
    { divide  $A[i], \dots, A[j]$  para que las claves menores que pivote estén a la
      izquierda y las claves mayores o iguales que pivote estén a la derecha.
      Devuelve el lugar donde se inicia el grupo de la derecha. }

    var
         $z, d$ : integer; { cursores, como se describieron antes }

    begin
        (1)       $z := i$ ;
        (2)       $d := j$ ;
                repeat
        (3)          intercambia( $A[z], A[d]$ );
                    { ahora se inicia la fase de rastreo }
        (4)          while  $A[z].clave < pivote$  do
        (5)               $z := z + 1$ ;
        (6)          while  $A[d].clave \geq pivote$  do
        (7)               $d := d - 1$ ;

                until
        (8)           $z > d$ ;
                return ( $z$ );
    end; { partición }

```

Fig. 8.13. Procedimiento *partición*.

Tiempo de ejecución de la clasificación rápida

Ahora se mostrará que el algoritmo lleva en promedio un tiempo $O(n \log n)$ para clasificar n elementos, y que en el peor caso lleva $O(n^2)$. El primer paso en la demostración de ambas proposiciones es probar qué *partición* lleva un tiempo proporcional al número de elementos que deberá separar, es decir, un tiempo $O(j - i + 1)$.

```

procedure quicksort (  $i, j$ : integer );
    { clasifica los elementos  $A[i], \dots, A[j]$  del arreglo externo  $A$  }
    var
         $pivot$ : tipo_clave; { el valor del pivote }
         $indice\_pivot$ : integer; { el índice de un elemento de  $A$  donde  $clave$  es
                                el  $pivot$  }
         $k$ : integer; { índice al inicio del grupo de elementos  $\geq pivot$  }
    begin
(1)       $indice\_pivot := encuentra\_pivot(i, j)$ ;
(2)      if  $indice\_pivot <> 0$  then begin { no hacer nada si todas las claves
                                son iguales }
(3)           $pivot := A[indice\_pivot].clave$ ;
(4)           $k := partición(i, j, pivot)$ ;
(5)          quicksort( $i, k - 1$ );
(6)          quicksort( $k, j$ )
        end
    end; { quicksort }

```

Fig. 8.14. El procedimiento *quicksort*.

Para ver por qué la proposición es cierta, es preciso usar un truco muy frecuente en el análisis de algoritmos: encontrar ciertos «artículos» a los cuales se les pueda «cargar» el tiempo, y mostrar cómo cargar cada paso del algoritmo para que ningún artículo se cargue más que alguna constante. El tiempo total consumido, entonces, no será mayor que el producto de esta constante por el número de artículos.

En este caso, los artículos son los elementos desde $A[i]$ hasta $A[j]$, y a cada uno se le carga todo el tiempo gastado por *partición* desde el momento en que z o d apunten hacia él por primera vez, hasta que dejan de hacerlo. Obsérvese primero que z y d nunca regresan a un elemento. A causa de que por lo menos existe un elemento en el grupo inferior y uno en el superior, y *partición* termina tan pronto como z excede a d , se sabe que cada elemento será cargado sólo una vez.

Un elemento se deja en los ciclos de las líneas (4) y (6) de la figura 8.13, ya sea por incremento de z o por decremento de d . ¿Cuánto tiempo puede transcurrir entre cada ejecución de $z := z + 1$ o $d := d - 1$? Lo peor que puede suceder es en el comienzo. Las líneas (1) y (2) asignan un valor inicial a z y d . Ahí se puede pasar por el ciclo sin hacer nada sobre z o d . En el segundo paso y en los siguientes, el intercambio de la línea (3) garantiza que los ciclos **while** de las líneas (4) y (6) se ejecutarán por lo menos una vez cada uno; así, lo peor que puede cargarse a una ejecución de $z := z + 1$ o $d := d - 1$ es el costo de las líneas (1), (2), dos veces (3), y las pruebas de

las líneas (4), (6), (8) y (4) otra vez. Esta es sólo una cantidad constante, independiente de i o j , y las siguientes ejecuciones de $z := z + 1$ o $d := d - 1$ se cargan menos: a lo sumo, una ejecución de las líneas (3) y (8) y un recorrido de los ciclos de las líneas (4) o (6).

Al final, también existen dos pruebas no satisfechas en las líneas (4), (6) y (8), que pueden no estar cargadas a ningún artículo, pero representan sólo una cantidad constante y pueden cargarse a cualquier artículo. Después de haber hecho todos los cargos, aún se tiene alguna constante c , por lo que ningún artículo ha sido cargado con más de c unidades de tiempo. Dado que existen $j - i + 1$ artículos, esto es, elementos en la porción del arreglo que se va a clasificar, el tiempo total empleado por *partición*(i, j, pivot) es $O(j - i + 1)$.

Ahora, considérese el tiempo de ejecución de *quicksort*(i, j). Es fácil comprobar que el tiempo consumido por llamada a *encuentra_pivot* de la línea (1) de la figura 8.14 es $O(j - i + 1)$, y en muchos casos es bastante menor. La prueba de la línea (2) requiere una cantidad constante de tiempo, al igual que el paso (3) cuando se ejecuta. Se ha mostrado que la línea (4), que llama a *partición*, llevará un tiempo $O(j - i + 1)$. Así, con excepción de las llamadas recursivas que hace a *quicksort*, cada llamada individual de *quicksort* lleva un tiempo como máximo proporcional al número de elementos que se le pide clasificar.

En otras palabras, el tiempo total consumido por *quicksort* es la suma, en todos los elementos, de las veces que el elemento forma parte del subarreglo en el que se hizo la llamada a *quicksort*. Recuerdese la figura 8.9, donde se observan las llamadas a *quicksort* organizadas por niveles. Es evidente que ningún elemento puede incluirse en dos llamadas del mismo nivel, así que el tiempo consumido por *quicksort* puede expresarse como la suma en todos los elementos de la *profundidad*, o máximo nivel, en el cual se encuentra ese elemento. Por ejemplo, los unos de la figura 8.9 son de profundidad 3 y el 6 es de profundidad 5.

En el peor caso, podría suceder que en cada llamada a *quicksort* se seleccionara el peor pivot posible, por ejemplo, el mayor valor de las claves en el subarreglo que se está clasificando. Entonces se dividiría el subarreglo en dos subarreglos más pequeños, uno con sólo un elemento (el elemento que tuviera al pivot como clave), y el otro con todos los demás. Esa secuencia de particiones forma un árbol como el de la figura 8.15, donde $r_1, r_2, \dots, r_n$ es la secuencia de registros en el orden creciente de las claves.

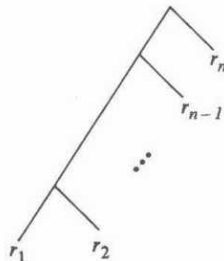


Fig. 8.15. Peor secuencia posible de selecciones de pivotes.

La profundidad de r_i es $n - i + 1$ para $2 \leq i \leq n$, y la profundidad de r_1 es $n - 1$. Así, la suma de las profundidades es

$$n - 1 + \sum_{i=2}^n (n - i + 1) = \frac{n^2}{2} + \frac{n}{2} - 1$$

la cual es $\Omega(n^2)$. En el peor caso, la clasificación rápida requiere un tiempo proporcional a n^2 para clasificar n elementos.

Análisis del caso promedio de la clasificación rápida

Como siempre, se interpreta «caso promedio» para un algoritmo de clasificación como el promedio sobre todas las clasificaciones iniciales, dando igual probabilidad a cualquier clasificación posible. Por simplicidad, se supondrá que no existen dos elementos con claves iguales. En general, las igualdades entre elementos hacen la tarea de clasificación más fácil, nunca más difícil.

Una segunda suposición que hace más fácil el análisis del algoritmo de clasificación rápida es que, cuando se llama a *quicksort*(i, j), todas las clasificaciones para $A[i], \dots, A[j]$ son igualmente probables. La justificación es que antes de la llamada, no había pivotes con los cuales $A[i], \dots, A[j]$ se pudieran comparar para distinguirlos entre sí; es decir, para que todos fueran menores que el pivote v , o para que todos fueran mayores. Una revisión cuidadosa del programa desarrollado muestra la probabilidad de que cada elemento pivote concluya cerca del extremo derecho del subarreglo de elementos mayores o iguales que él, pero para subarreglos grandes, el hecho de que el elemento mínimo (el pivote previo) pueda aparecer cerca del extremo derecho no marca una diferencia considerable †.

Ahora, sea $T(n)$ el tiempo promedio consumido por la clasificación rápida para ordenar n elementos. Es evidente que $T(1)$ es alguna constante c_1 , ya que en un elemento esta clasificación no hace llamadas recursivas a sí mismas. Cuando $n > 1$, como se supone que todos los elementos tienen claves distintas, se sabe que la clasificación rápida tomará un pivote y dividirá el subarreglo, consumiendo un tiempo $c_2 n$ para hacerlo, para alguna constante c_2 , y después llamará a la clasificación en los dos subarreglos. Sería bueno poder pedir que el pivote tuviera la misma probabilidad de ser el primero, segundo, ..., n -ésimo elemento en el orden clasificado, para el subarreglo que se esté ordenando. Sin embargo, para garantizar que la clasificación rápida encuentre por lo menos una clave menor que cada pivote y al menos una igual o mayor que el pivote (de modo que cada fragmento sea menor que el total y, por tanto, no sean posibles los ciclos infinitos), siempre se escoge el mayor de los dos primeros elementos encontrados. Resulta que esta selección no afecta a la distribución de tamaños de los subarreglos, pero tiende a hacer los grupos izquierdos (aquellos que son menores que el pivote) más grandes que los grupos derechos.

† Si hay una razón para creer que los ordenamientos no aleatorios de elementos pueden hacer que *quicksort* se ejecute más lentamente de lo esperado, el programa *quicksort* debería permutar aleatoriamente los elementos del arreglo antes de la clasificación.

Se hará el desarrollo de una fórmula para la probabilidad de que el grupo izquierdo tenga i de los n elementos, en el supuesto de que todos los elementos son distintos. Para que el grupo izquierdo tenga i elementos, el pivote debe ser el $(i + 1)$ -ésimo elemento entre los n . El pivote, por el método de selección, podía haber estado en la primera posición, con alguno de los i menores como segundo, o bien pudo haber sido el segundo, con uno de los i menores como primero. La probabilidad de que cualquier elemento en particular, tal como el $(i + 1)$ -ésimo, aparezca primero en una secuencia aleatoria, es $1/n$. Dado que apareció primero, la probabilidad de que el segundo elemento sea uno de los i elementos más pequeños, de los $n - 1$ elementos restantes es $i/(n - 1)$. Así, la probabilidad de que el pivote aparezca en la primera posición y sea el número $i + 1$ de los n en el orden apropiado, es $i/n(n - 1)$. En forma semejante, la probabilidad de que el pivote aparezca en la segunda posición y sea el número $i + 1$ de los n en el orden clasificado es $i/n(n - 1)$, así que la probabilidad de que el grupo izquierdo sea de tamaño i es $2i/n(n - 1)$, para $1 \leq i < n$.

Ahora, se puede escribir una ecuación de recurrencia para $T(n)$.

$$T(n) \leq \sum_{i=1}^{n-1} \frac{2i}{n(n-1)} [T(i) + T(n-i)] + c_2 n \quad (8.1)$$

Según la ecuación (8.1), el tiempo promedio requerido por la clasificación rápida es el tiempo, $c_2 n$, empleado fuera de las llamadas recursivas, más el tiempo promedio utilizado en las mismas. Este último tiempo se expresa en (8.1) como la suma, en todas las posibles i , de la probabilidad de que el grupo izquierdo tenga tamaño i (y, por tanto, el grupo derecho tiene tamaño $n - i$), multiplicado por el costo de las dos llamadas recursivas: $T(i)$ y $T(n - i)$, respectivamente.

La primera tarea es transformar la ecuación (8.1) de manera que se simplifique la sumatoria y, de hecho, tome la forma que tendría si se hubiera elegido un pivote verdaderamente aleatorio en cada paso. Para hacer la transformación, obsérvese que para una función $f(i)$ cualquiera, sustituyendo i por $n - i$ se puede probar

$$\sum_{i=1}^{n-1} f(i) = \sum_{i=1}^{n-1} f(n-i) \quad (8.2)$$

Al sustituir la mitad del lado izquierdo de (8.2) por el lado derecho, queda

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{2} \sum_{i=1}^{n-1} (f(i) + f(n-i)) \quad (8.3)$$

Aplicando (8.3) a (8.1) con $f(i)$ igual a la expresión interna de la sumatoria de (8.1), se obtiene

$$\begin{aligned} T(n) &\leq \frac{1}{2} \sum_{i=1}^{n-1} \left\{ \frac{2i}{n(n-1)} [T(i) + T(n-i)] + \frac{2(n-i)}{n(n-1)} [T(n-i) + T(i)] \right\} + c_2 n \\ &\leq \frac{1}{n-1} \sum_{i=1}^{n-1} [T(i) + T(n-i)] + c_2 n \end{aligned} \quad (8.4)$$

A continuación, se aplica (8.3) a (8.4), con $f(i) = T(i)$. Esta transformación da

$$T(n) \leq \frac{2}{n-1} \sum_{i=1}^{n-1} T(i) + c_2 n \quad (8.5)$$

Obsérvese que (8.4) es la ecuación de recurrencia que se obtendría si todos los tamaños entre 1 y $n-1$ para el grupo izquierdo fueran igualmente probables. Así, tomar como pivote la mayor de dos claves en realidad no afecta a la distribución de tamaños. Se estudiarán recurrencias de esta forma con mayor detalle en el capítulo 9. Aquí se resolverá la recurrencia (8.5) proponiendo una solución y demostrando que funciona. La solución propuesta es que $T(n) \leq cn \log n$ para alguna constante c y toda $n \geq 2$.

Para demostrar que esta suposición es correcta, se efectúa una inducción sobre n . Para $n=2$, sólo se observa que para alguna constante c , $T(2) \leq 2c \log 2 = 2c$. Para comprobar la inducción, se supone que $T(i) \leq ci \log i$ para $i < n$, y se sustituye esta fórmula por $T(i)$ en el lado derecho de (8.5) para demostrar que la cantidad resultante no es mayor que $cn \log n$. Así, (8.5) se convierte en

$$T(n) \leq \frac{2c}{n-1} \sum_{i=1}^{n-1} i \log i + c_2 n \quad (8.6)$$

Aquí es necesario dividir la sumatoria de (8.6) en términos menores, donde $i \leq n/2$, y, por tanto, $\log i$ no es mayor que $\log(n/2)$, que es $(\log n) - 1$, y en términos mayores, donde $i > n/2$, y $\log i$ puede ser tan grande como $\log n$. Entonces (8.6) se convierte en

$$\begin{aligned} T(n) &\leq \frac{2c}{n-1} \left[\sum_{i=1}^{n/2} i \log i + \sum_{i=n/2+1}^{n-1} i \log i \right] + c_2 n \\ &\leq \frac{2c}{n-1} \left[\sum_{i=1}^{n/2} i (\log n - 1) + \sum_{i=n/2+1}^{n-1} i \log n \right] + c_2 n \\ &\leq \frac{2c}{n-1} \left[\frac{n}{4} \left(\frac{n}{2} + 1 \right) \log n - \frac{n}{4} \left(\frac{n}{2} + 1 \right) + \frac{3}{4} n \left(\frac{n}{2} - 1 \right) \log n \right] + c_2 n \\ &\leq \frac{2c}{n-1} \left[\left(\frac{n^2}{2} - \frac{n}{2} \right) \log n - \left(\frac{n^2}{8} + \frac{n}{4} \right) \right] + c_2 n \\ &\leq cn \log n - \frac{cn}{4} - \frac{cn}{2(n-1)} + c_2 n \end{aligned} \quad (8.7)$$

Al tomar $c \geq 4c_2$, la suma del segundo y cuarto términos de (8.7) no es mayor que cero. El tercer término de (8.7) hace una contribución negativa, así que de (8.7) es posible confirmar que $T(n) \leq cn \log n$, si $c = 4c_2$. Esto completa la demostración de que la clasificación rápida requiere un tiempo $O(n \log n)$ en el caso promedio.

Mejoras a la clasificación rápida

Quicksort es muy rápido, su tiempo promedio de ejecución es menor que el de todos los algoritmos de clasificación $O(n \log n)$ conocidos en la actualidad (en un factor constante, por supuesto). Es factible mejorar aún más el factor constante al tomar pivotes que dividan cada subarreglo en partes similares. Por ejemplo, al dividir siempre los subarreglos en partes iguales, cada elemento será de profundidad exactamente $\log n$, en el árbol de particiones semejante al de la figura 8.9. En comparación, la profundidad promedio de un elemento para *quicksort*, como se constituyó en la figura 8.14, es de cerca de $1.4 \log n$. Así, cabe esperar un incremento en la velocidad de *quicksort* seleccionando los pivotes con cuidado.

Por ejemplo, se pueden escoger tres elementos de un subarreglo al azar y tomar el elemento medio como pivote. Se pueden tomar k elementos al azar para cualquier k , clasificarlos por una llamada recursiva a la clasificación rápida o por uno de los algoritmos más simples de la sección 8.2, y elegir el elemento medio, es decir, el elemento $[(k+1)/2]$ -ésimo como pivote †. Es un ejercicio interesante determinar el mejor valor de k como una función del número de elementos del subarreglo a clasificar. Si k es muy pequeña, se malgasta el tiempo, porque, en promedio, el pivote dividirá los elementos de forma desigual. Si k es muy grande, llevará demasiado tiempo encontrar el elemento medio de los k elementos.

Otra mejora de la clasificación rápida está relacionada con lo que sucede cuando se toman subarreglos pequeños. Recuérdese de la sección 8.2 que los métodos simples $O(n^2)$ son mejores que los métodos $O(n \log n)$ para n pequeñas. La pequeñez de n depende de muchos factores, como el tiempo empleado en una llamada recursiva, la cual es una propiedad de la arquitectura de la máquina y de la estrategia utilizada por el compilador para realizar las llamadas a procedimientos en el lenguaje en que se escribió el método de clasificación. Knuth [1973] sugiere 9 como el tamaño del subarreglo en el que *quicksort* debe llamar a un algoritmo de clasificación más simple.

Existe otra forma de «acelerar» *quicksort*, que en realidad es una forma de canjear espacio por tiempo; la misma idea es válida para cualquier otro algoritmo de clasificación. Si se tiene espacio disponible, se crea un arreglo de apuntadores a los registros del arreglo A . Se efectúan las comparaciones entre las claves de los registros apuntados, pero sin mover los registros; en vez de eso, se mueven los apuntadores a los registros de la misma forma que la clasificación rápida mueve los registros. Al final, los apuntadores, leídos de izquierda a derecha, apuntan a los registros

† Dado que sólo se desea la mediana y no la lista completa clasificada de k elementos, puede ser mejor usar uno de los algoritmos de la sección 8.7, que encuentran la mediana con rapidez.

en el orden deseado, y será relativamente fácil reordenar los registros de A en el orden correcto.

De esta forma, sólo se hacen n intercambios de registros, en lugar de $O(n \log n)$, lo cual significa una diferencia sustancial si los registros son grandes. Por el lado negativo, se requiere espacio adicional para el arreglo de apuntadores, y el acceso a las claves para efectuar las comparaciones es más lento que antes, ya que se debe seguir primero el apuntador, y luego ir al registro, para conseguir el campo de la clave.

8.4 Clasificación por montículos (*heapsort*)

En esta sección se desarrolla un algoritmo de clasificación llamado *clasificación por montículos* (*heapsort*) cuyo peor caso y el caso promedio son $O(n \log n)$. Este algoritmo puede expresarse en forma abstracta por medio de las cuatro operaciones de conjuntos INSERTA, SUPRIME, VACIA y MIN, presentadas en los capítulos 4 y 5. Supóngase que L es la lista de elementos que se va a clasificar y S es un conjunto de elementos de tipo `tipo_registro` que se usará para guardar los elementos conforme se clasifican. El operador MIN se aplica al campo de la clave de los registros; esto es, $\text{MIN}(S)$ devuelve el registro en S cuya clave tiene el valor más pequeño. La figura 8.16 presenta el algoritmo de clasificación abstracto que se transformará en una clasificación por montículos.

```
(1)  for  $x$  en la lista  $L$  do
(2)      INSERTA( $x$ ,  $S$ );
(3)  while not VACIA( $S$ ) do begin
(4)       $y := \text{MIN}(S)$ ;
(5)      writeln( $y$ );
(6)      SUPRIME( $y$ ,  $S$ )
end
```

Fig. 8.16. Algoritmo abstracto de clasificación.

Los capítulos 4 y 5 analizaron varias estructuras de datos, como los árboles 2-3, que manejan cada operación en un tiempo $O(\log n)$ por operación, si los conjuntos nunca crecen más allá de n elementos. Si se supone que la lista L es de longitud n , el número de operaciones realizadas será n veces INSERTA, n veces SUPRIME, n veces MIN, y $n + 1$ pruebas VACIA. El tiempo total consumido por el algoritmo de la figura 8.16 es $O(n \log n)$, si se emplea una estructura de datos adecuada.

La estructura de datos de árbol parcialmente ordenado que se estudió en la sección 4.11, es adecuada para la realización de este algoritmo. Recuerdese que un árbol parcialmente ordenado puede representarse por un *montículo*, un arreglo $A[1], \dots, A[n]$, cuyos elementos tienen la propiedad de árbol parcialmente ordenado: $A[i].\text{clave} \leq A[2*i].\text{clave}$ y $A[i].\text{clave} \leq A[2*i + 1].\text{clave}$. Al considerar los elementos $2i$ y $2i + 1$ como los «hijos» del elemento en i , el arreglo forma un árbol binario equilibrado en el cual la clave del padre nunca excede a las claves de los hijos.

En la sección 4.11, se vio que el árbol parcialmente ordenado puede manejar las

operaciones INSERTA y SUPRIME_MIN en un tiempo $O(\log n)$ por operación. Mientras que el árbol parcialmente ordenado no puede manejar la operación general SUPRIME en un tiempo $O(\log n)$ (sólo encontrar un elemento arbitrario lleva un tiempo lineal en el peor caso), debe hacerse notar que en la figura 8.16, los únicos elementos que se eliminan son los encontrados como minimales. Así, las líneas (4) y (6) de la figura 8.16 pueden combinarse en una función SUPRIME_MIN que devuelve el elemento y . Con esto se puede obtener el algoritmo de la figura 8.16 con la estructura de datos árbol parcialmente ordenado de la sección 4.11.

Es necesaria una modificación más al algoritmo de la figura 8.16 para evitar imprimir los elementos conforme se eliminan. El conjunto S siempre estará almacenado como un montículo en la parte superior del arreglo A , como $A[1], \dots, A[i]$ si S tiene i elementos. Por la propiedad de árbol parcialmente ordenado, el elemento más pequeño estará siempre en $A[1]$. Los elementos que se van eliminando de S pueden almacenarse en $A[i+1], \dots, A[n]$, clasificados en orden inverso, es decir, con $A[i+1] \geq A[i+2] \geq \dots \geq A[n]$ †. Como $A[1]$ debe ser el menor de $A[1], \dots, A[i]$, puede efectuarse la operación SUPRIME_MIN simplemente intercambiando $A[1]$ con $A[i]$. Ya que el nuevo $A[i]$ (el anterior $A[1]$) no es menor que $A[i+1]$ (o el anterior se habría eliminado de S antes que el último), se tienen $A[i], \dots, A[n]$ clasificados en orden decreciente. Se puede considerar que S se encuentra ocupando $A[1], \dots, A[i-1]$.

Dado que el $A[1]$ nuevo ($A[i]$ anterior) viola la propiedad de árbol parcialmente ordenado, debe descender en el árbol como en el procedimiento SUPRIME_MIN de la figura 4.23. Aquí se utiliza el procedimiento *empuja*, que se muestra en la figura 8.17, que opera sobre el arreglo A definido en forma externa. Mediante una secuencia de intercambios, *empuja* lleva el elemento $A[\text{primero}]$ hasta su lugar adecuado entre sus descendientes en el árbol. Para restaurar la propiedad de árbol parcialmente ordenado en el montículo, *empuja* se llama con $\text{primero} = 1$.

Las líneas (4) a (6) de la figura 8.16 funcionan de la siguiente manera. La selección del mínimo en la línea (4) es fácil: siempre se encuentra en $A[1]$ gracias a la propiedad de árbol parcialmente ordenado. En vez de imprimir en la línea (5), se intercambia $A[1]$ con $A[i]$, el último elemento del montículo actual. Esto simplifica la eliminación del elemento mínimo en el árbol parcialmente ordenado; sólo se disminuye i , el cursor que indica el fin del montículo actual. Entonces, se invoca *empuja* (1, i) para restituir la propiedad de árbol parcialmente ordenado al montículo, $A[1], \dots, A[i]$.

La prueba de ausencia de elementos de S realizada en la línea (3) de la figura 8.16, se hace probando el valor de i , el cursor que marca el fin del montículo actual. Sólo resta ahora considerar cómo trabajan las líneas (1) y (2). Se puede suponer que L está presente originalmente en $A[1], \dots, A[n]$, en algún orden dado. Para establecer inicialmente la propiedad de árbol ordenado, se llama *empuja* (j , n) para toda $j = n/2, n/2-1, \dots, 1$. Obsérvese que después de las llamadas a *empuja* (j , n), no se viola la propiedad de árbol parcialmente ordenado en $A[j], \dots, A[n]$, porque al empujar en el árbol un registro no se introducen nuevas violaciones, pues sólo se inter-

† Al final, se podría invertir el arreglo A , pero si se desea que A termine clasificado de menor a mayor, simplemente se aplica un operador SUPRIME_MAX en lugar de SUPRIME_MIN, y se ordena A parcialmente, de modo que un padre no tenga claves menores (en vez de mayores) que sus hijos.

cambia un registro violador con su hijo más pequeño. El procedimiento completo, llamado *heapsort*, se muestra en la figura 8.18.

Análisis de la clasificación por montículos

Examinéase ahora el procedimiento *empuja* para saber el tiempo que requiere. Una inspección de la figura 8.17 confirma que el cuerpo del ciclo **while** lleva un tiempo constante. Además, después de cada iteración, r tiene por lo menos el doble del valor que tenía. Así, puesto que r empieza igual a *primero*, después de i iteraciones se tiene, $r \geq \text{primero} * 2^i$. Es seguro que $r > \text{último}/2$ si $\text{primero} * 2^i > \text{último}/2$, esto es si

$$i > \log (\text{último}/\text{primero}) - 1 \quad (8.8)$$

```

procedure empuja (primero, último: integer);
  {supone que  $A[\text{primero}]$ , ...,  $A[\text{último}]$  obedece la propiedad de árbol
   parcialmente ordenado, excepto, quizá, para el hijo de  $A[\text{primero}]$ . El
   procedimiento empuja  $A[\text{primero}]$  hasta que se restituye la propiedad
   de árbol parcialmente ordenado }

  var
     $r$ : integer; { indica la posición actual de  $A[\text{primero}]$  }

  begin
     $r := \text{primero}$ ; { asignación de valores iniciales }
    while  $r \leq \text{último} \div 2$  do
      if  $\text{último} = 2 * r$  then begin {  $r$  tiene un hijo en  $2 * r$  }
        if  $A[r].\text{clave} > A[2 * r].\text{clave}$  then
          intercambia ( $A[r]$ ,  $A[2 * r]$ );
           $r := \text{último}$  { fuerza la terminación del ciclo while }
        end
      else {  $r$  tiene dos hijos, los elementos ubicados en  $2 * r$  y  $2 * r + 1$  }
        if  $A[r].\text{clave} > A[2 * r].\text{clave}$  and
           $A[2 * r].\text{clave} \leq A[2 * r + 1].\text{clave}$  then begin
            { intercambia  $r$  con su hijo izquierdo }
            intercambia( $A[r]$ ,  $A[2 * r]$ );
             $r := 2 * r$ 
          end
        else if  $A[r].\text{clave} > A[2 * r + 1].\text{clave}$  and
           $A[2 * r + 1].\text{clave} < A[2 * r].\text{clave}$  then begin
            { intercambia  $r$  con su hijo derecho }
            intercambia( $A[r]$ ,  $A[2 * r + 1]$ );
             $r := 2 * r + 1$ 
          end
        else {  $r$  no viola la propiedad de árbol parcialmente
          ordenado }
           $r := \text{último}$  { para forzar la terminación del ciclo while }
      end; { empuja }
  end;

```

Fig. 8.17. El procedimiento *empuja*.

Aquí, el número de iteraciones del ciclo **while** en *empuja* es $\log(\text{último}/\text{primero})$ a lo sumo.

Dado que $\text{primero} \geq 1$ y $\text{último} \leq n$ en cada llamada que el algoritmo de la figura 8.18 hace a *empuja*, (8.8) dice que cada llamada a *empuja*, en la línea (2) o (5) de la figura 8.18, lleva un tiempo $O(\log n)$. Es evidente que el ciclo de las líneas (1) y (2) se ejecuta $n/2$ veces, así que el tiempo dedicado es $O(n \log n)$ †. También el ciclo de las líneas (3) a (5) se ejecuta $n - 1$ veces. Así un tiempo total $O(n)$ se consume en todas las repeticiones de *intercambia* en la línea (4), y $O(n \log n)$ durante las repeticiones de la línea (5). Así, el tiempo total gastado en el ciclo de las líneas (3) a (5) es $O(n \log n)$, y todo *heapsort* lleva un tiempo $O(n \log n)$.

```

procedure heapsort;
    { clasifica el arreglo A[1],...,A[n] en orden decreciente }

    var
        i: integer; { cursor hacia A }

    begin
        { establece inicialmente la propiedad de árbol
          parcialmente ordenado }
    (1)   for i := n div 2 downto 1 do
    (2)       empuja(i,n);
    (3)   for i:= n downto 2 do begin
    (4)       intercambia(A[1], a[i]);
           { elimina el mínimo del frente del montículo }
    (5)       empuja(1, i-1)
           { restablece la propiedad de árbol parcialmente ordenado }

        end
    end; { heapsort }

```

Fig. 8.18. El procedimiento *heapsort*.

A pesar de su tiempo $O(n \log n)$ en el peor caso, *heapsort* llevará en promedio más tiempo que *quicksort*, en un pequeño factor constante. La clasificación por montículos tiene interés intelectual porque es el primer algoritmo $O(n \log n)$ en el peor caso que se ha estudiado. Es de utilidad práctica cuando no se desea clasificar los n elementos, sino sólo las k menores de entre ellos, con k mucho menor que n . Como se mencionó antes, las líneas (1) a (2) en realidad sólo requieren un tiempo $O(n)$. Si se realizan sólo k iteraciones de las líneas (3) a (5), el tiempo empleado en

† De hecho, este tiempo es $O(n)$, por un argumento más cuidadoso. Para j dentro del intervalo $n/2$ a $n/4 + 1$, (8.8) dice que sólo se necesita una iteración en el ciclo **while** de *empuja*. Para j entre $n/4$ y $n/8 + 1$, sólo dos iteraciones, y así sucesivamente. El número total de iteraciones cuando j está comprendida entre $n/2$ y 1 está acotado por $n/4 \cdot 1 + n/8 \cdot 2 + n/16 \cdot 3 + \dots$. Obsérvese que el límite mejorado para las líneas (1) a (2) no implica un límite mejorado para *heapsort*; todo el tiempo se consume en las líneas (3) a (5).

ese ciclo es $O(k \log n)$. Así, *heapsort*, modificado para producir sólo los primeros k elementos, lleva un tiempo $O(n + k \log n)$. Si $k \leq n/\log n$, es decir, se desea como máximo una fracción $(1/\log n)$ de la lista completa clasificada, entonces el tiempo requerido es $O(n)$.

8.5 Clasificación por urnas (*binsort*)

Se plantea la cuestión de si son necesarios $\Omega(n \log n)$ pasos para clasificar n elementos. En la siguiente sección se mostrará que ése es el caso de los algoritmos de clasificación que no suponen algo acerca del tipo de datos de las claves, excepto que pueden ordenarse mediante alguna función que indica si el valor de alguna clave es «menor que» otro. En muchas ocasiones es posible clasificar en tiempos menores a $O(n \log n)$, siempre que se conozca algo especial acerca de las claves que se están clasificando.

Ejemplo 8.5. Supóngase que *tipo_clave* es entero, y que se sabe que los valores de las claves se encuentran en el intervalo de 1 a n , sin duplicados, donde n es el número de elementos. Entonces, si A y B son del tipo `array [1..n] of tipo_registro`, y los n elementos a clasificar están almacenados inicialmente en A , es posible colocarlos en orden en el arreglo B , por medio de

```
for i := 1 to n do
    B[A[i]. clave] := A[i];
```

(8.9)

Este código calcula el lugar que pertenece al registro $A[i]$ y lo coloca en él. El ciclo completo lleva un tiempo $O(n)$. Trabaja bien sólo cuando existe un único registro con clave v para todo valor de v entre 1 y n . Un segundo registro que tenga la clave v puede ubicarse también en $B[v]$, destruyendo el registro anterior con clave v .

Hay otras formas de clasificar en su sitio un arreglo A con claves 1, 2, ..., n en sólo un tiempo $O(n)$. Se visitan $A[1]$, ..., $A[n]$ por turno; si el registro de $A[i]$ tiene clave $j \neq i$, se intercambian $A[i]$ con $A[j]$. Si después del intercambio el registro con la clave k se encuentra en $A[i]$, y $k \neq i$, se intercambia $A[i]$ con $A[k]$, y así sucesivamente. Cada intercambio coloca algún registro donde le corresponde, después de lo cual no vuelve a moverse. Así, el siguiente algoritmo ordena A en su sitio, en un tiempo $O(n)$, a condición de que exista un registro con cada una de las claves 1, 2, ..., n .

```
for i := 1 to n do
    while A[i]. clave <> i do
        intercambia (A[i], A[A[i]. clave]);
```

□

El programa (8.9) dado en el ejemplo 8.5 es un caso simple de una «clasificación por urnas» (*binsort*), un proceso de clasificación donde se crea una urna para contener todos los registros con cierta clave. Se examina cada registro r a clasificar y se

coloca en la urna de acuerdo con el valor de la clave de r . En el programa (8.9), las urnas son los elementos del arreglo $B[1], \dots, B[n]$, y $B[i]$ es la urna para la clave cuyo valor es i . Se pueden usar elementos del arreglo como urnas en este caso simple, porque se sabe que nunca habrá más de un elemento en una urna. Más aún, no es necesario ensamblar las urnas en una lista clasificada, porque B sirve como tal lista.

En el caso general, sin embargo, a veces puede ser necesario almacenar más de un registro en una urna y enlazarlas (o *concatenarlas*) en el orden apropiado. En otras palabras, supóngase que, como siempre, $A[1], \dots, A[n]$ es un arreglo del tipo `tipo_registro`, y que las claves de los registros son del tipo `tipo_clave`. Sólo a efectos de esta sección, se supondrá que `tipo_clave` es un tipo enumerado, tal como $1..m$ o *char*. Sea `tipo_lista` un tipo que representa listas de elementos de tipo `tipo_registro`; `tipo_lista` puede ser cualquiera de los tipos para listas mencionados en el capítulo 2, pero una lista enlazada es más efectiva, ya que se generarán listas de tamaño impredecible en cada urna; con todo, las longitudes totales de las listas estarán fijadas en n , y, por tanto, un arreglo de n celdas puede proporcionar las listas para las urnas según sea necesario.

Por último, sea B un arreglo del tipo `array[tipo_clave] of tipo_lista`. Entonces, B es un arreglo de urnas, que son listas (o, si se usa la representación enlazada de listas, encabezado de listas). B está indizada por `tipo_clave`, así que existe una urna para cada posible valor de clave. De esta forma se puede efectuar la primera generalización de (8.9): las urnas tienen capacidad arbitraria.

Ahora, es necesario considerar cómo se concatenarán las urnas. De manera abstracta, partiendo de las listas $a_1, a_2, \dots, a_i$ y $b_1, b_2, \dots, b_j$, debe formarse la *concatenación*, que será $a_1, a_2, \dots, a_i, b_1, b_2, \dots, b_j$. La realización de esta operación $\text{CONCATENA}(L_1, L_2)$, que reemplaza la lista L_1 por la concatenación $L_1 L_2$, puede efectuarse en cualquiera de las representaciones de listas estudiadas en el capítulo 2.

Sin embargo, por eficiencia, es útil tener un apuntador al último elemento en cada lista (o al encabezado si la lista se encuentra vacía), además de uno de encabezamiento. Esta modificación facilita la búsqueda del último elemento en la lista L_1 sin

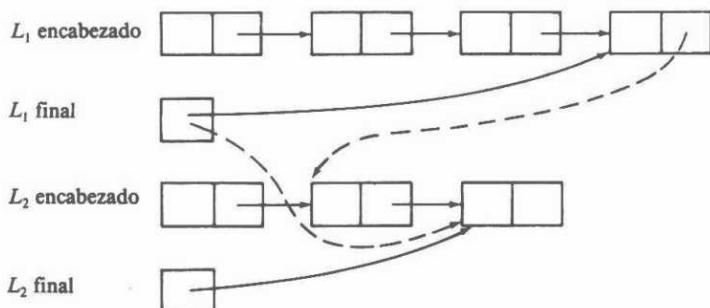


Fig. 8.19. Concatenación de listas enlazadas.

necesidad de recorrer toda la lista. La figura 8.19 muestra los apuntadores revisados, con líneas de puntos necesarios para concatenar L_1 y L_2 y obtener el resultado L_1 . Se supone que la lista L_2 «desaparece» después de la concatenación, en el sentido de que el encabezado y el apuntador final de L_2 se vuelven nulos.

Ahora se puede escribir un programa para clasificar colecciones arbitrarias de registros por medio de urnas, donde el campo de la clave es de un tipo enumerado. El programa de la figura 8.20 está escrito en función de primitivas de procesamiento de listas. Como ya se ha mencionado, una lista enlazada es la aplicación preferible, pero existen otras opciones. Recuérdese también que el ambiente para el procedimiento es que un arreglo A de tipo `array[1..n] of tipo_registro` contiene los elementos a clasificar, y el arreglo B , de tipo `array[tipo_clave] of tipo_lista` representa las urnas. Se supone que `tipo_clave` es expresable como `clave_menor..clave_mayor`, como debe ser cualquier tipo enumerado, para algunas cantidades `clave_menor` y `clave_mayor`.

```

procedure clasificación_por_urnas;
  { clasifica el arreglo  $A$  mediante urnas, dejando la lista ordenada en
     $B[\text{clave\_menor}]$  }
  var
     $i$ : integer;
     $v$ : tipo_clave;
  begin
    { coloca los registros en las urnas }
  (1)   for  $i := 1$  to  $n$  do
        { mete  $A[i]$  al frente de la urna destinada a su clave }
  (2)   INSERTA( $A[i]$ , PRIMERO( $B[A[i].clave]$ ),  $B[A[i].clave]$ );
  (3)   for  $v := \text{succ}(\text{lowkey})$  to  $\text{clave\_mayor}$  do
        { concatena todas las urnas al final de  $B[\text{clave\_menor}]$  }
  (4)   CONCATENA( $B[\text{clave\_menor}]$ ,  $B[v]$ )
  end; { clasificación_por_urnas }

```

Fig. 8.20. Programa abstracto de clasificación por urnas.

Análisis de la clasificación por urnas

Se pretende que si hay n elementos a clasificar y m valores distintos de claves (y, por tanto, m urnas distintas), el programa de la figura 8.20 lleva un tiempo $O(n + m)$, si la estructura de datos empleada para las urnas es la adecuada. En particular, si $m \leq n$, la clasificación por urnas lleva un tiempo $O(n)$. La estructura de datos en cuestión es una lista enlazada. Los apuntadores a los finales de las listas, como se indica en la figura 8.19, son útiles, pero no indispensables.

El ciclo de las líneas (1) a (2) de la figura 8.20, que coloca los registros en las urnas, lleva un tiempo $O(n)$, debido a que la operación INSERTA de la línea (2) requiere un tiempo constante, pues la inserción siempre se hace al principio de la lista. Para

el ciclo de las líneas (3) a (4), que concatena las urnas, se supone temporalmente que existen apuntadores a los finales de las listas. El paso (4) emplea un tiempo constante, por lo que el ciclo lleva un tiempo $O(m)$. De aquí que el programa de clasificación por urnas completo lleve un tiempo $O(n + m)$.

Si los apuntadores a los finales de las listas no existen, en la línea (4) se debe consumir cierto tiempo para llegar al final de $B[v]$ antes de la concatenación con $B[\text{clave_menor}]$. De esta manera, el extremo de $B[\text{clave_menor}]$ siempre estará disponible para la siguiente concatenación. El tiempo adicional dedicado a llegar al final de cada urna, suma un tiempo de $O(n)$, debido a que la suma de las longitudes de las urnas es n . Este tiempo extra no afecta al orden de magnitud del tiempo de ejecución para el algoritmo, debido a que $O(n)$ no es mayor que $O(n + m)$.

Clasificación de grandes conjuntos de claves

Si m , el número de claves, no es mayor que n , el número de elementos, el tiempo de ejecución $O(n + m)$ del procedimiento de la figura 8.20 es en realidad $O(n)$. Pero ¿qué sucedería si $m = n^2$? Evidentemente, la figura 8.20 necesitará $O(n + n^2)$, lo cual es $O(n^2)$. ¿Será posible aprovechar el hecho de que el conjunto de claves está limitado para optimar el algoritmo? La respuesta sorprendente es que aun si el conjunto de claves posibles es $1, 2, \dots, n^k$, para cualquier k fija, existe una generalización de la técnica de clasificación por urnas, que requiere un tiempo de sólo $O(n)$.

Ejemplo 8.6. Considérese el problema específico de clasificación de n enteros en el intervalo de 0 a $n^2 - 1$. Se clasificarán los n enteros en dos fases; la primera parece que no es de mucha ayuda, pero es esencial. Se emplean n urnas, una para cada entero entre 0 y $n - 1$. Se coloca cada entero i de la lista a clasificar dentro de la urna numerada $i \bmod n$. Sin embargo, a diferencia de la figura 8.20, es importante agregar cada entero al final de la lista de la urna, no al principio. Si se quiere que la agregación sea eficiente, hace falta la representación de listas enlazadas para cada urna con apuntadores a los finales de las listas.

Por ejemplo, supóngase que $n = 10$, y que la lista a clasificar está constituida por los cuadrados perfectos desde 0^2 hasta 9^2 en el orden aleatorio 36, 9, 0, 25, 1, 49, 64, 16, 81, 4. En este caso, donde $n = 10$, la urna para el entero i sólo es el dígito más a la derecha de i escrito en decimal. La figura 8.21(a) muestra la colocación de esta lista en las urnas. Obsérvese que los enteros aparecen en las urnas en el mismo orden en el cual aparecen en la lista original; por ejemplo, la urna 6 contiene 36, 16, y no 16, 36, debido a que 36 precede a 16 en la lista original.

Ahora, se concatenan en orden las urnas, produciendo la lista

$$0, 1, 81, 64, 4, 25, 36, 16, 9, 49 \quad (8.10)$$

de la figura 8.21(a). Si se utiliza la estructura de datos de lista enlazada con apuntadores a los finales de las listas, la colocación de los n -enteros en las urnas y su concatenación pueden llevar un tiempo $O(n)$ cada una.

En la lista creada por la concatenación de las urnas, los enteros se redistribuyen en urnas, pero con una estrategia de selección diferente. Ahora se coloca el entero i dentro de la urna $[i/n]$, esto es, el máximo entero menor o igual que i/n . De nuevo, los enteros se agregan al final de las listas de las urnas. Al concatenar en orden las urnas, se observa que la lista está clasificada.

En el presente ejemplo, la figura 8.21 (b) muestra la lista (8.10) distribuida en las urnas con i contenido en la urna $[i/10]$.

Para ver por qué este algoritmo funciona, hay que observar que cuando varios enteros se colocan en una urna, como sucedió con 0, 1, 4 y 9, que se colocaron en la urna 0, deben estar en orden creciente, ya que la lista (8.10) resultante del primer recorrido los ordenó de acuerdo con el dígito más a la derecha. Así, en cualquier urna los dígitos de más a la derecha deben formar una secuencia creciente. Por supuesto, cualquier entero colocado en la urna i debe preceder a un entero colocado en una urna mayor que i , y al concatenar en orden las urnas se produce la lista ordenada.

<i>Urna</i>	<i>Contenido</i>	<i>Urna</i>	<i>Contenido</i>
0	0	0	0, 1, 4, 9
1	1, 81	1	16
2		2	25
3		3	36
4	64, 4	4	49
5	25	5	
6	36, 16	6	64
7		7	
8		8	81
9	9, 49	9	

(a)
(b)

Fig. 8.21. Clasificación por urnas en dos recorridos.

En forma más general, pueden considerarse los enteros entre 0 y $n^2 - 1$ como números de dos dígitos con base n y usar el mismo argumento para comprobar que la estrategia de clasificación funciona. Sean los enteros $i = an + b$ y $j = cn + d$, donde a, b, c y d se encuentran en el intervalo 0 a $n - 1$; esto es, son dígitos de base n . Supóngase que $i < j$, entonces $a > c$ no es posible, y debe suponerse que $a \leq c$. Si $a < c$, i aparece en una urna menor que j después del segundo recorrido, por lo que i precederá a j en el orden final. Si $a = c$, b debe ser menor que d . Después del primer recorrido, i precede a j , ya que i fue colocado en la urna b , y j , en la d . Así, aunque i y j se colocan en la misma urna a (la mitad que c), i se inserta antes que j en la urna. □

Clasificación general por residuos (*radix sort*)

Supóngase que tipo_clave es una secuencia de campos, como en

```

type
  tipo_clave = record
    día : 1..31;
    mes : (ene,...,dic);
    año : 1900..1999;
  end;

```

(8.11)

o un arreglo de elementos del mismo tipo, como en

```

type
  tipo_clave = array[1..10] of char;

```

(8.12)

Se supondrá de aquí en adelante que tipo_clave está constituido por k elementos, $f_1, f_2, \dots, f_k$ de tipos $t_1, t_2, \dots, t_k$. Por ejemplo, en (8.11) $t_1 = 1..31$, $t_2 = (\text{ene}, \dots, \text{dic})$, y $t_3 = 1900..1999$. En (8.12), $k = 10$, y $t_1 = t_2 = \dots = t_k = \text{char}$.

También se supondrá de que se desea clasificar registros en orden *lexicográfico* de acuerdo con sus claves. Esto es, el valor de la clave $(a_1, a_2, \dots, a_k)$ es menor que el valor de la clave $(b_1, b_2, \dots, b_k)$, donde a_i y b_i son los valores del campo f_i , para $i = 1, 2, \dots, k$, si

1. $a_1 < b_1$, o bien
2. $a_1 = b_1$ y $a_2 < b_2$, o bien

·
·
·

- k . $a_1 = b_1, a_2 = b_2, \dots, a_{k-1} = b_{k-1}$, y $a_k < b_k$.

Esto es, para alguna j entre 0 y $k-1$, $a_1 = b_1, \dots, a_j = b_j$, y $a_{j+1} < b_{j+1}$.

Se pueden considerar las claves del tipo antes definido como si los valores de las claves fueran enteros expresados en alguna notación de residuos extraña. Por ejemplo, (8.12), donde cada campo es un carácter, puede considerarse como la expresión de enteros en base 128 o de tantos caracteres como haya en el conjunto de caracteres de la máquina empleada. La definición de tipo (8.11) puede considerarse como si el lugar de más a la derecha estuviera en base 100 (correspondiente a los valores entre 1900 y 1999), el siguiente lugar en base 12, y el tercero en base 31. Desde este punto de vista, la clasificación por urnas generalizada se conoce como clasificación *por residuos* (*radix sorting*). En último caso, se puede emplear para clasificar enteros hasta cualquier límite fijo, tomándolos como arreglos de dígitos de base 2 u otra.

La idea clave de la clasificación por residuos es ordenar por urnas todos los registros, primero en f_k , el «dígito menos significativo», después concatenar las urnas,

† Obsérvese que una secuencia que va desde 1 hasta 0 (o más generalmente, desde x hasta y , donde $y < x$) se considera una secuencia vacía.

primero el menor valor, clasificar de nuevo en f_{k-1} , y así sucesivamente. Como en el ejemplo 8.6, al insertar en las urnas hay que asegurarse de que cada registro se agregue al final de la lista, no al principio. El algoritmo de clasificación por residuos se esboza en la figura 8.22; la razón de su funcionamiento se ilustró en el ejemplo 8.6. En general, después de la clasificación por urnas en $f_k, f_{k-1}, \dots, f_1$, los registros aparecerán en orden lexicográfico si las claves constan sólo de los campos $f_1, \dots, f_k$.

Análisis de la clasificación por residuos

Primero se debe emplear la estructura de datos adecuada para hacer una clasificación eficiente. Obsérvese que se supone que la lista a clasificar ya está en forma de lista enlazada, no de arreglo. En la práctica, sólo es necesario agregar un campo adicional, el campo de enlace, al tipo tipo_registro, para poder enlazar $A[i]$ a $A[i + 1]$ para $i = 1, 2, \dots, n - 1$ y así hacer una lista enlazada del arreglo A en un tiempo $O(n)$. Obsérvese también que si se presentan en esta forma los elementos a clasificar, nunca se copia un registro. Sólo se cambian registros de una lista a otra.

```

procedure radixsort;
{ clasifica la lista  $A$  de  $n$  registros con claves que consisten en campos
 $f_1, \dots, f_k$  de tipos  $t_1, \dots, t_k$  respectivamente. El procedimiento usa los
arreglos  $B_i$  de tipo array  $[t_i]$ , of tipo_lista para  $1 \leq i \leq k$ , donde
tipo_lista es una lista enlazada de registros. }
begin
(1)   for  $i := k$  downto 1 do begin
(2)       for cada valor  $v$  de tipo  $t_i$  do { limpia las urnas }
(3)           vaciar  $B_i[v]$ 
(4)       for cada registro  $r$  de la lista  $A$  do
(5)           mover  $r$  desde  $A$  hasta el final de la urna  $B_i[v]$ , donde  $v$  es
el valor del campo  $f_i$  de la clave de  $r$ 
(6)       for cada valor  $v$  de tipo  $t_i$ , desde el menor hasta el mayor do
(7)           concatena  $B_i[v]$  en el extremo de  $A$ 
end
end; { radixsort }

```

Fig. 8.22. Clasificación por residuos.

Como antes, para hacer la concatenación con rapidez, se necesitan apuntadores al final de cada lista. Después, el ciclo de las líneas (2) y (3) de la figura 8.22 lleva un tiempo $O(s_i)$, donde s_i es el número de diferentes valores del tipo t_i . El ciclo de las líneas (4) y (5) lleva un tiempo $O(n)$, y el de las líneas (6) y (7), $O(s_i)$. Así, el

tiempo total requerido por la clasificación por residuos es $\sum_{i=1}^k O(s_i + n)$, lo cual es $O(kn + \sum_{i=1}^k s_i)$, u $O(n + \sum_{i=1}^k s_i)$, si se toma k como una constante.

Ejemplo 8.7. Si las claves son enteros en el intervalo 0 a $n^k - 1$, para alguna constante k , se puede generalizar el ejemplo 8.6 y considerar las claves como enteros en base n con k dígitos de longitud. Entonces, t_i es $0..(n-1)$ para toda i entre 1 y k , así que $s_i = n$. La expresión $O(n + \sum_{i=1}^k s_i)$ se vuelve $O(n + kn)$ que, como k es una constante, es $O(n)$.

Como otro ejemplo, si las claves son cadenas de caracteres de longitud k , para la constante k , entonces $s_i = 128$, por ejemplo, para toda i , y $\sum_{i=1}^k s_i$ es una constante. Así, la clasificación por residuos en cadenas de caracteres de longitud fija también toma $O(n)$. De hecho, siempre que k es constante y los s_i son constantes, o simplemente $O(n)$, la clasificación por residuos lleva un tiempo $O(n)$. Sólo si k crece con n , puede ocurrir que no tome $O(n)$. Por ejemplo, si las claves se consideran como cadenas binarias de longitud $\log n$, entonces $k = \log n$, y $s_i = 2$ para $1 \leq i \leq k$. Así, la clasificación por residuos tomaría $O(kn + \sum_{i=1}^k s_i)$, lo cual es $O(n \log n)$ †. □

8.6 Cota inferior para la clasificación por comparaciones

Existe un «teorema heurístico» que dice que la clasificación de n elementos «requiere un tiempo $n \log n$ ». Se estudió en la sección anterior que esta proposición no siempre es cierta; si el tipo de la clave es tal que la clasificación por urnas o la clasificación por residuos puedan usarse con ventaja, el tiempo $O(n)$ es suficiente. Sin embargo, estos algoritmos de clasificación se basan en claves de tipos especiales: un tipo con un conjunto limitado de valores. Todos los demás algoritmos de clasificación estudiados cuentan sólo con el hecho de que se puede probar si una clave es menor que otra.

Se debe tener en cuenta que en todos los algoritmos de clasificación anteriores a la sección 8.5, la determinación del orden apropiado de los elementos se hace al comparar dos claves, de modo que el flujo de control del algoritmo siga uno de los dos caminos. En contraste, un algoritmo como el del ejemplo 8.5 hace que uno de n diferentes eventos suceda sólo en un paso, al almacenar un registro con una clave entera en una de las n urnas, dependiendo del valor de ese entero. Todos los programas de la sección 8.5 usan una posibilidad de los lenguajes de programación y de las máquinas que es mucho más poderosa que una simple comparación de valores; es la posibilidad de encontrar en un paso una localidad de un arreglo, dado el índice de esa localidad. Pero este poderoso tipo de operación no es factible si tipo-clave fuera, por ejemplo, real. No es posible, en Pascal ni en muchos otros lenguajes, declarar un arreglo indizado por números reales, y si lo fuera, no se podrían concatenar, en una cantidad de tiempo razonable, todas las urnas correspondientes a los números reales representables en la máquina.

† Pero en este caso, si los enteros de $\log n$ bits pueden ocupar una palabra, será mejor tratar las claves como si estuvieran compuestas de un solo campo, de tipo $1..n$, usando una clasificación por urnas ordinaria.

Arboles de decisión

Se tratarán ahora los algoritmos de clasificación que sólo usan los elementos a clasificar cuando comparan dos claves. Se puede dibujar un árbol binario en el cual los nodos representan el «estado» del programa después hacer un número de comparaciones de claves. También se puede considerar que un nodo representa las disposiciones iniciales de los elementos que llevarán el programa a este «estado». Así, un «estado» del programa es, en esencia, el conocimiento de la disposición inicial conseguida hasta ese punto por el programa.

Si cualquier nodo representa dos o más disposiciones iniciales posibles, el programa todavía no puede conocer el orden correcto, por lo que debe hacer otra comparación de claves, como «¿es $k_1 < k_2$?». Entonces se pueden crear dos hijos para el nodo; el hijo izquierdo representa aquellas disposiciones iniciales consistentes con el hecho de $k_1 < k_2$, y el hijo derecho representa las disposiciones consistentes con el hecho de que $k_1 > k_2$ †. Así, cada hijo representa un «estado» que contiene la información conocida en el padre, más el hecho de que $k_1 < k_2$ o que $k_1 > k_2$, dependiendo de si el hijo es izquierdo o derecho.

Ejemplo 8.8. Considérese el algoritmo de clasificación por inserción con $n = 3$. Supóngase que al principio, $A[1]$, $A[2]$ y $A[3]$ tienen claves con valores a , b y c , respectivamente. Cualquiera de las seis disposiciones de a , b y c puede ser la correcta, así que se empieza la construcción del árbol de decisión con el nodo etiquetado (1) de la figura 8.23, el cual representa todas las posibles disposiciones. El algoritmo de clasificación por inserción compara primero $A[2]$ con $A[1]$, esto es, b con a . Si b resultara ser el más pequeño, la disposición correcta sólo puede ser bac , bca o cba , las tres disposiciones en las cuales b precede a a . Esas tres disposiciones están representadas por el nodo (2) de la figura 8.23. Las otras tres disposiciones están representadas por el nodo (3), el hijo derecho de (1), y son las disposiciones para las cuales a precede a b .

Ahora considérese lo que sucede si el dato inicial es tal que se alcanza el nodo (2). Sólo se ha intercambiado $A[1]$ con $A[2]$, y se encontró que $A[2]$ no puede subir más ya que se encuentra en la «cima». La disposición actual de los elementos es bac . La clasificación por inserción empieza a continuación insertando $A[3]$ en el lugar adecuado, y comparando $A[3]$ con $A[2]$. Dado que $A[2]$ contiene ahora a , y $A[3]$ contiene c , y comparando c con a ; el nodo (4) representa las dos disposiciones del nodo (2) en las cuales c precede a a , mientras que (5) representa la disposición donde esto no suceda.

El algoritmo terminará si alcanza el estado del nodo (5), ya que ha movido a $A[3]$ lo más arriba posible. Por otra parte, en el estado del nodo (4), se tiene que $A[3] < A[2]$, por lo que se intercambiaron, dejando b en $A[1]$ y c en $A[2]$. La clasificación por inserción compara esos dos elementos y los intercambia si a continuación $c < b$. Los nodos (8) y (9) representan las disposiciones consistentes con $c < b$ y su opuesto, respectivamente, así como la información recogida de los nodos (1) a

† También es posible suponer que todas las claves son diferentes, ya que si se clasifica una colección de claves distintas, con seguridad se producirá un orden correcto cuando alguna clave esté repetida.

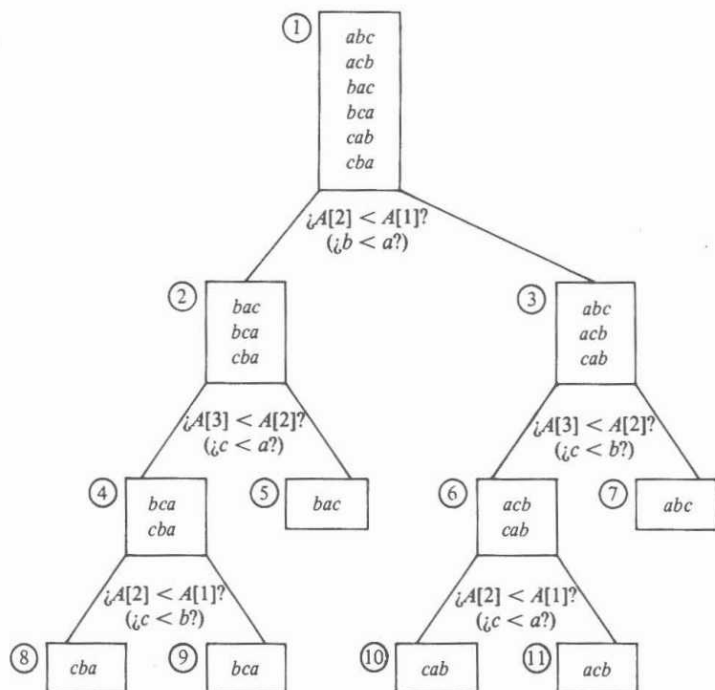


Fig. 8.23. Árbol de decisión para la clasificación por inserción con $n = 3$.

(2) a (4), es decir, $b < a$ y $c < a$. La clasificación por inserción termina, se haya intercambiado $A[2]$ con $A[1]$ o no, y no se efectúan más comparaciones. Por fortuna, cada hoja (5), (8) y (9) representa disposiciones individuales, así que se ha acumulado suficiente información para determinar la clasificación correcta.

La descripción del árbol que desciende del nodo (3) es bastante simétrica con lo que se ha visto, por lo que se omite. La figura 8.23 se emplea como guía para determinar el orden correcto de las llaves a , b y c , puesto que todas sus hojas están asociadas con una disposición individual.

Tamaño de los árboles de decisión

La figura 8.23 tiene seis niveles, que corresponden a las seis posibles disposiciones de la lista inicial a , b , c . En general, si se clasifica una lista de n elementos, existen $n! = n(n-1)(n-2)\dots(2)(1)$ resultados posibles, los cuales se encuentran en el orden

correcto para la lista inicial $a_1, a_2, \dots, a_n$. Esto es, cualquiera de los n elementos puede estar en primer lugar, cualquiera de los $n-1$ elementos puede estar en segundo lugar, cualquiera de los $n-2$ puede estar en tercer lugar, y así sucesivamente. De esta forma, cualquier árbol de decisión que describa un algoritmo de clasificación correcto y que trabaje en una lista de n elementos, debe tener por lo menos $n!$ hojas, ya que cada disposición posible debe estar sola en una hoja. De hecho, si se eliminan nodos correspondientes a comparaciones innecesarias y hojas que no corresponden a disposiciones posibles (ya que las hojas pueden alcanzarse sólo por una serie inconsistente de resultados de comparaciones), habrá exactamente $n!$ hojas.

Los árboles binarios con muchas hojas deben tener caminos largos. La longitud de un camino desde la raíz hasta una hoja proporciona el número de comparaciones hechas cuando la disposición representada por esa hoja es el orden clasificado para cierta lista de entrada L . Así, la longitud del camino más largo desde la raíz hasta una hoja es una cota inferior en el número de pasos efectuados por el algoritmo en el peor caso. En especial, si L es la lista de entrada, el algoritmo efectuará al menos tantas comparaciones como la longitud del camino, probablemente en adición a otros pasos que no sean comparaciones de claves.

Por tanto, es necesario preguntarse lo cortos que pueden ser todos los caminos en un árbol binario con k hojas. Un árbol binario en el que todos los caminos son de longitud p o menor, pueden tener una raíz, dos nodos en el nivel 1, cuatro nodos en el nivel 2 y, en general, 2^i nodos en el nivel i . Así, el número mayor de hojas de un árbol sin nodos en los niveles más altos que p es 2^p . Es decir, un árbol binario con k hojas debe tener un camino de longitud no menor que $\log k$. Si $k = n!$, entonces cualquier algoritmo de clasificación que sólo utilice comparaciones para determinar el orden clasificado, en el peor caso debe requerir un tiempo $\Omega(\log(n!))$.

Pero ¿con qué rapidez crece $\log(n!)$? Una aproximación cercana a $n!$ es $(n/e)^n$, donde $e = 2.7183\dots$ es la base de los logaritmos naturales. Dado que $\log((n/e)^n) = n \log n - n \log e$, se observa que $\log(n!)$ es de orden $n \log n$. Es posible tomar una cota inferior precisa observando que $n(n-1)\dots(2)(1)$ es el producto de por lo menos $n/2$ factores, que a su vez son cada uno al menos $n/2$. Así, $n! \geq (n/2)^{n/2}$; de aquí que $\log(n!) \geq (n/2)\log(n/2) = (n/2)\log n - n/2$, por lo que la clasificación por comparaciones requiere un tiempo $\Omega(n \log n)$ en el peor caso.

Análisis del caso promedio

¿Puede haber algún algoritmo que sólo use comparaciones para clasificar, y requiera un tiempo $\Omega(n \log n)$ en el peor caso, como todos los algoritmos de este tipo, pero que el tiempo promedio requerido sea $O(n)$ o algo menor que $O(n \log n)$? La respuesta es no, y sólo se mencionará cómo probar la aseveración, dejando los detalles al lector.

Lo que se desea probar es que en cualquier árbol binario con k hojas, la profundidad promedio de una hoja es por lo menos $\log k$. Supóngase que no fuera así, y el árbol T fuera el contraejemplo con menos nodos. T no puede ser un solo nodo, porque la aseveración dice que los árboles de una hoja tienen profundidad promedio

por lo menos de 0. Ahora, supóngase que T tiene k hojas. Un árbol binario con $k \geq 2$ hojas es semejante a los árboles de las figuras 8.24(a) o (b).

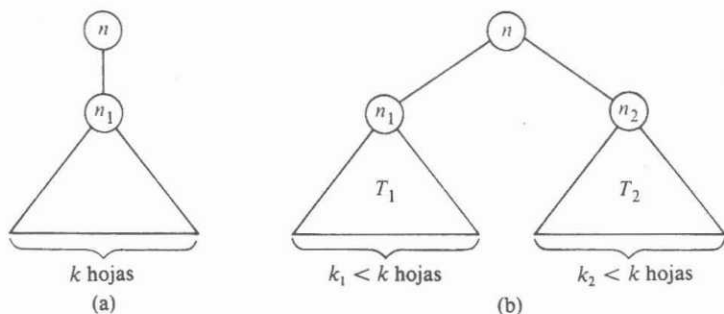


Fig. 8.24. Formas posibles del árbol binario T .

La figura 8.24(a) no puede ser el contraejemplo más pequeño, porque el árbol cuya raíz es n_1 tiene tantas hojas como T , pero una profundidad promedio más pequeña. Si la figura 8.24(b) fuera T , los árboles cuyas raíces son n_1 y n_2 , al ser más pequeños que T , no violarían la suposición. Esto es, la profundidad promedio de las hojas de T_1 es al menos $\log(k_1)$, y la profundidad promedio de T_2 es por lo menos $\log(k_2)$. Entonces, la profundidad promedio de T es

$$\left(\frac{k_1}{k_1 + k_2} \right) \log(k_1) + \left(\frac{k_2}{k_1 + k_2} \right) \log(k_2) + 1.$$

Como $k_1 + k_2 = k$, la profundidad promedio se expresa como

$$\frac{1}{k} (k_1 \log(2k_1) + k_2 \log(2k_2)) \quad (8.13)$$

Se puede comprobar que cuando $k_1 = k_2 = k/2$, (8.13) tiene valor $k \log k$, y se debe demostrar que (8.13) tiene un mínimo cuando $k_1 = k_2$, dada la restricción $k_1 + k_2 = k$. Se deja esta demostración como ejercicio a quien tenga experiencia en cálculo diferencial. Si se asume que (8.13) tiene un valor mínimo de $k \log k$, se observa que T no fue un contraejemplo.

8.7 Estadísticas de orden

El problema del cálculo de *estadísticas de orden* consiste en encontrar la clave del k -ésimo registro en el orden clasificado de los registros, dada una lista de n registros y un entero k . En general, este problema se plantea como «encontrar el k -ésimo en-

tre n . Ocurren casos especiales cuando $k = 1$ (encontrar el mínimo), $k = n$ (encontrar el máximo), y el caso en que n es impar y $k = (n + 1)/2$, que es encontrar la mediana.

Ciertos casos del problema son muy fáciles de resolver en tiempo lineal. Por ejemplo, encontrar el mínimo de n elementos en un tiempo $O(n)$ no requiere nada especial. Como se mencionó en relación con la clasificación por montículos, si $k \leq n/\log n$, es posible encontrar el k -ésimo de n construyendo un montículo, que lleva un tiempo $O(n)$, y después seleccionar los k elementos más pequeños en un tiempo $O(n + k \log n) = O(n)$. Sistemáticamente, se puede encontrar el k -ésimo de n en un tiempo $O(n)$ cuando $k \geq n - n/\log n$.

Una variante de la clasificación rápida

Tal vez la forma más rápida para encontrar el k -ésimo entre n , en promedio, es usar un procedimiento recursivo similar a la clasificación rápida, que se puede llamar *selecciona* (i, j, k), y que encuentra el k -ésimo elemento entre $A[i], \dots, A[j]$ dentro de un arreglo más grande $A[i], \dots, A[n]$. Los pasos básicos de *selecciona* son:

1. Tomar un elemento pivote, como v .
2. Usar el procedimiento *partición* de la figura 8.13 para dividir $A[i], \dots, A[j]$ en dos grupos: $A[i], \dots, A[m - 1]$ con claves menores que v , y $A[m], \dots, A[j]$ con claves v o mayores.
3. Si $k \leq m - i$, el k -ésimo entre $A[i], \dots, A[j]$ está en el primer grupo, se llama a *selecciona* ($i, m - 1, k$); si $k > m - i$, se llama a *selecciona* ($m, j, k - m + i$).

Con el tiempo se encuentra que se llama a *selecciona* (i, j, k), cuando todos los elementos $A[i], \dots, A[j]$ tienen la misma clave (en general, porque $j = i$). Entonces se sabe que la clave deseada es cualquiera de las encontradas en esos registros.

Como con la clasificación rápida, la función *selecciona* descrita antes puede llevar un tiempo $\Omega(n^2)$ en el peor caso. Por ejemplo, supóngase que se busca el primer elemento, pero por mala suerte el pivote siempre es la mayor clave disponible. Sin embargo, en promedio, *selecciona* es aún más rápido que la clasificación rápida; lleva un tiempo $O(n)$. El motivo es que mientras esta última se llama a sí misma dos veces, *selecciona* lo hace sólo una. Se puede analizar *selecciona* como se hizo con la clasificación rápida, pero de nuevo las matemáticas son complejas, y un simple concepto intuitivo debe ser convincente. En promedio, *selecciona* se llama a sí mismo en un subarreglo la mitad de largo que el subarreglo dado. Supóngase que, en forma conservadora, se dice que cada llamada es en un arreglo cuyo tamaño es $9/10$ de la llamada previa. Entonces, si $T(n)$ es el tiempo empleado por *selecciona* en un arreglo de longitud n , se sabe que para alguna constante c ,

$$T(n) \leq T\left(\frac{9}{10}n\right) + cn \quad (8.14)$$

Usando técnicas del próximo capítulo, se puede mostrar que la solución de (8.14) es $T(n) = O(n)$.

Método lineal en el peor caso para encontrar estadísticas de orden

Para garantizar que una función como *selecciona* tenga en el peor caso, en vez de en el caso promedio, complejidad $O(n)$, es suficiente demostrar que en un tiempo lineal se puede encontrar algún pivote que con certeza esté a una fracción positiva de la distancia desde cualquier extremo. Por ejemplo, la solución a (8.14) muestra que si el pivote de n elementos nunca es menor que el elemento $(n/10)$ -ésimo, ni mayor que el elemento $(9n/10)$ -ésimo, de forma que la llamada recursiva a *selecciona* se da como máximo en nueve décimas del arreglo, esta variante de *selecciona* será $O(n)$ en el peor caso.

La clave para encontrar un buen pivote está contenida en los dos pasos siguientes.

1. Dividir los n elementos en grupos de cinco, dejando a un lado entre 0 y 4 elementos que no puedan colocarse en un grupo. Clasificar cada grupo de cinco con cualquier algoritmo y tomar el elemento central de cada grupo, hasta un total de $\lfloor n/5 \rfloor$ elementos.
2. Usar *selecciona* para encontrar la mediana de esos $\lfloor n/5 \rfloor$ elementos, o si $\lfloor n/5 \rfloor$ es par, un elemento en la posición más cercana al centro. Tanto si $\lfloor n/5 \rfloor$ es par como si es impar, el elemento deseado estará en la posición $\lfloor (n+5)/10 \rfloor$.

Este pivote está lejos de los extremos, siempre que no sean muchos los registros que tengan el pivote como clave $\dagger$. Para simplificar, se supondrá por el momento que todas las claves son diferentes. Entonces, se dice que el pivote elegido, el elemento $\lfloor (n+5)/10 \rfloor$ -ésimo de los $\lfloor n/5 \rfloor$ elementos centrales obtenidos de los grupos de cinco, es mayor que al menos $3\lfloor (n-5)/10 \rfloor$ de los n elementos, puesto que excede $\lfloor (n-5)/10 \rfloor$ de los elementos centrales, y cada uno de éstos excede dos más, de los cinco de los cuales es el centro. Si $n \geq 75$, $3\lfloor (n-5)/10 \rfloor$ es por lo menos $n/4$. En forma semejante, se puede probar que el pivote escogido es menor o igual que al menos $3\lfloor (n-5)/10 \rfloor$ elementos, así que para $n \geq 75$, el pivote cae entre los puntos $1/4$ y $3/4$ en el orden clasificado. Más importante, cuando se escoge el pivote para dividir los n elementos, el elemento k -ésimo se aislará dentro de un intervalo de hasta $3n/4$ de los elementos. Un esbozo del algoritmo completo se proporciona en la figura 8.25; igual que para los algoritmos de clasificación, éste supone un arreglo $A[i], \dots, A[n]$ de tipo_registro, y ese tipo_registro tiene un campo *clave* del tipo tipo_clave. El algoritmo para encontrar el k -ésimo elemento es sólo una llamada a *selecciona* (1, n , k).

Para analizar el tiempo de ejecución de *selecciona* de la figura 8.25, sea $n_j = i + 1$. Las líneas (2) y (3) se ejecutan sólo si n es 74 o menor. Así, aunque el paso (2) en general puede llevar $O(n^2)$ pasos, hay una constante c_1 , con independencia de su tamaño, tal que para $n \leq 74$, las líneas (1) a (3) no llevan más de un tiempo c_1 .

$\dagger$ En el caso extremo, cuando todas las claves son iguales, el pivote no ofrece ninguna separación. Obviamente, el pivote es el k -ésimo elemento para cualquier k , y se hace necesario otro enfoque.

```

function selecciona ( i, j, k: integer ) : tipo_clave;
    { devuelve la clave del  $k$ -ésimo elemento de acuerdo con el orden
      de clasificación entre  $A[i], \dots, A[j]$  }
    var
        m: integer; { utilizado como índice }
    begin
        (1)   if  $j-i < 74$  then begin { muy pocos para usar selecciona recursivamente }
        (2)       clasifica  $A[i], \dots, A[j]$  mediante algún algoritmo simple;
        (3)       return ( $A[i+k-1].clave$ )
                end
        else begin { aplica selecciona recursivamente }
        (4)       for  $m := 0$  to  $(j-i-4) \text{ div } 5$  do
                    { toma los elementos medios de los grupos de cinco
                      en  $A[i], A[i+1], \dots$  }
        (5)       encuentra el tercer elemento entre  $A[i+5*m]$  y
                     $A[i+5*m+4]$  y lo intercambia con  $A[i+m]$ ;
        (6)        $pivot := selecciona(i, i+(j-i-4) \text{ div } 5, (j-i-4) \text{ div } 10);$ 
                    { encuentra la mediana de los elementos medios. Obsérvese
                      que  $j-i-4$  aquí es  $n-5$  de la descripción informal
                      anterior }
        (7)        $m := partición(i, j, pivot);$ 
        (8)       if  $k \leq m-i$  then
        (9)         return ( $selecciona(i, m-1, k)$ )
                else
        (10)        return( $selecciona(m, j, k-(m-i))$ )
                end
    end; { selecciona }

```

Fig. 8.25. Algoritmo lineal en el peor caso para encontrar el k -ésimo elemento.

Ahora, considérese las líneas (4) a (10). La línea (7), el paso de la partición, se mostró en conexión con la clasificación rápida, y lleva tiempo $O(n)$. El ciclo de las líneas (4) a (5) se ejecuta unas $n/5$ veces, y cada ejecución de la línea (5), requiriendo la clasificación de 5 elementos, lleva un tiempo constante, de manera que el ciclo lleva un tiempo $O(n)$ en total.

Sea $T(n)$ el tiempo requerido por una llamada a *selecciona* con n elementos. Entonces, la línea (6) lleva como máximo un tiempo $T(n/5)$. Ya que $n \geq 75$ cuando se alcanza la línea (10), y se ha especificado que si $n \geq 75$, a lo sumo $3n/4$ elementos son menores que el pivote, y hasta $3n/4$ son iguales o mayores, se sabe que las líneas (9) o (10) requieren como máximo un tiempo $T(3n/4)$. Así, para algunas constantes c_1 y c_2 ,

$$T(n) \leq \begin{cases} c_1 & \text{si } n \leq 74 \\ c_2 n + T(n/5) + T(3n/4) & \text{si } n \geq 75 \end{cases} \quad (8.15)$$

El término c_2n de (8.15) representa las líneas (1), (4), (5) y (7); el término $T(n/5)$ se deriva de la línea (6), y $T(3n/4)$ representa las líneas (9) y (10).

Se mostrará que $T(n)$ de (8.15) es $O(n)$. Antes de proseguir, debe apreciarse que el «número mágico» 5, el tamaño de los grupos de la línea (5), y la elección de $n = 75$ en el punto de equilibrio debajo del cual no se usa *selecciona* en forma recursiva, fueron diseñados para que los argumentos $n/5$ y $3n/4$ de T en (8.15) sumaran algo menos que n . Se podrían hacer otras elecciones de esos parámetros, pero obsérvese que al resolver (8.15) se necesita saber que $1/5 + 3/4 < 1$ para probar la linealidad.

La ecuación (8.15) puede resolverse suponiendo una solución y verificando por inducción que se cumple para toda n . Se seleccionará una solución de la forma cn , para alguna constante c . Si se escoge $c \geq c_1$, es sabido que $T(n) \leq cn$ para toda n entre 1 y 74, así que se considerará el caso $n \geq 75$. Supóngase por inducción que $T(m) \leq cm$ para $m < n$. Entonces, por (8.15),

$$T(n) \leq c_2n + cn/5 + 3cn/4 \leq c_2n + 19cn/20 \quad (8.16)$$

Al elegir $c = \max(c_1, 20c_2)$, por (8.16) se tiene $T(n) \leq cn/20 + cn/5 + 3cn/4 = cn$, lo cual se pretendía demostrar. Así, $T(n)$ es $O(n)$.

Caso en el que existen algunas igualdades entre claves

Recuérdese que en la figura 8.25 se supuso que no había dos claves iguales. La razón de esta suposición es que en otro caso no puede mostrarse que la línea (7) divide A en bloques de tamaño $3n/4$ como máximo. La modificación requerida para manipular igualdades entre claves es agregar, después del paso (7), otro paso tipo partición, que agrupe todos los registros con claves iguales al pivote. Por ejemplo, existen $p \geq 1$ de estas claves. Si $m - i \leq k \leq m - i + p$, entonces la recursión no es necesaria; simplemente se devuelve $A[m]$. clave. De otra forma, la línea (8) no cambia, pero la línea (10) llama a *selecciona*($m + p, j, k - (m - i) - p$).

Ejercicios

- 8.1 Dados los ocho enteros 1, 7, 3, 2, 0, 5, 0, 8. Clasifíquense por medio de a) clasificación de burbuja, b) clasificación por inserción, y c) clasificación por selección.
- 8.2 Dados los dieciséis enteros 22, 36, 6, 79, 26, 45, 75, 13, 31, 62, 27, 76, 33, 16, 62, 47. Clasifíquense usando a) clasificación rápida, b) clasificación por inserción, c) clasificación por montículos y d) clasificación por urnas, tratándolos como pares de dígitos en el intervalo 0 - 9.
- 8.3 El procedimiento *Shellsort* (clasificación de Shell) de la figura 8.26, algunas veces llamado *clasificación de incremento decreciente*, clasifica un arreglo $A[1..n]$ de enteros, al clasificar $n/2$ pares $(A[i], A[n/2 + i])$ para $1 \leq i \leq n/2$ en el primer recorrido, $n/4$ cuádruplos $(A[i], A[n/4 + i], A[n/2 + i],$

$A[3n/4 + i]$) para $1 \leq i \leq n/4$ en el segundo recorrido, $n/8$ óctuplos en el tercer recorrido, y así sucesivamente. En cada recorrido, el ordenamiento se realiza con la clasificación por inserción, la cual termina cuando encuentra dos elementos en el orden apropiado.

```

procedure Shellsort ( var A: array[1..n] of integer );
var
    i, j, incr: integer;
begin
    incr := n div 2;
    while incr > 0 do begin
        for i := incr + 1 to n do begin
            j := i - incr;
            while j > 0 do
                if A[j] > A[j+incr] then begin
                    intercambia(A[j], A[j+incr]);
                    j := j - incr;
                end
                else
                    j := 0 { fuerza la terminación del ciclo }
                end;
            incr := incr div 2;
        end
    end; { Shellsort }

```

Fig. 8.26. Clasificación de Shell (*Shellsort*).

- a) Clasifíquense las secuencias de enteros de los ejercicios 8.1 y 8.2 usando *Shellsort*.
 - *b) Muéstrase que si $A[i]$ y $A[n/2^k + i]$ quedan clasificados en el recorrido k (es decir, fueron intercambiados), entonces esos dos elementos permanecen clasificados en el recorrido $k + 1$.
 - c) Las distancias entre elementos comparados e intercambiados en un recorrido, disminuyen como $n/2, n/4, \dots, 2, 1$ en la figura 8.26. Demuéstrase que *Shellsort* trabajará con cualquier secuencia de distancias siempre que la última distancia sea 1.
 - **d) Muéstrase que *Shellsort* trabaja en un tiempo $O(n^{1.5})$.
- 8.4** Supóngase que se está clasificando una lista L que consta de una lista clasificada seguida de unos cuantos elementos «aleatorios». ¿Cuál de los métodos de clasificación analizados en este capítulo será especialmente apto para tal tarea?
- *8.5** Un algoritmo de clasificación es *estable* si conserva el orden original de los registros cuyas claves son iguales. ¿Cuáles de los algoritmos de clasificación de este capítulo son estables?

- *8.6** Supóngase que se emplea una variante de la clasificación rápida, donde siempre se elige como pivote el primer elemento del subarreglo que se está clasificando.
- ¿Qué modificaciones deben hacerse al algoritmo de la figura 8.11 para evitar ciclos infinitos cuando haya una secuencia de elementos iguales?
 - Demuéstrese que el algoritmo modificado tiene un tiempo de ejecución de $O(n \log n)$ en el caso promedio.
- 8.7** Muéstrese que cualquier algoritmo de clasificación que mueva los elementos sólo una posición a la vez, debe tener una complejidad de tiempo de $\Omega(n^2)$ al menos.
- 8.8** En la clasificación por montículos, el procedimiento *empuja* de la figura 8.17 establece la propiedad de árbol parcialmente ordenado en tiempo $O(n)$. En vez de empezar en las hojas empujando elementos hasta formar un montículo, se podría empezar en la raíz y empujar elementos hacia arriba. ¿Cuál es la complejidad de tiempo de este método?
- *8.9** Supóngase que se tiene un conjunto de palabras, por ejemplo, cadenas de letras a a z , cuya longitud total es n . Muéstrese cómo clasificar esas palabras en un tiempo $O(n)$. Obsérvese que si la longitud máxima de las palabras es constante, funcionará la clasificación por urnas. Sin embargo, debe considerarse el caso en que algunas palabras sean muy largas.
- *8.10** Muéstrese que el tiempo de ejecución de la clasificación por inserción es $\Omega(n^2)$ en el caso promedio.
- **8.11** Considérese el siguiente algoritmo *clasif_aleatoria* para clasificar un arreglo $A[1..n]$ de enteros: si los elementos $A[1], A[2], \dots, A[n]$ están en el orden adecuado, se para; en otro caso, selecciona un número aleatorio i entre 1 y n , intercambia $A[1]$ y $A[i]$, y se repite. ¿Cuál es el tiempo de ejecución esperado para *clasif_aleatoria*?
- *8.12** Previamente se mostró que la clasificación por comparaciones requiere $\Omega(n \log n)$ comparaciones en el peor caso. Demuéstrese que esta cota inferior persiste también en el caso promedio.
- *8.13** Pruébese que el procedimiento *selecciona*, descrito de manera informal al principio de la sección 8.7, tiene un tiempo de ejecución $O(n)$ en el caso promedio.
- 8.14** Obténgase CONCATENA para la estructura de datos de la figura 8.19.
- 8.15** Escribese un programa para encontrar los k elementos más pequeños de un arreglo de longitud n . ¿Cuál es la complejidad de tiempo del programa? ¿Para qué valor de k es ventajoso clasificar el arreglo?
- 8.16** Escribese un programa para encontrar los elementos mayor y menor de un arreglo. ¿Puede hacerse esto con menos de $2n - 3$ comparaciones?

- 8.17 Escribese un programa para encontrar el elemento más frecuente de una lista de elementos. ¿Cuál es la complejidad de tiempo del programa?
- *8.18 Muéstrase que cualquier algoritmo para eliminar duplicados de una lista requiere por lo menos un tiempo $\Omega(n \log n)$ con el modelo de computación del árbol de decisión de la sección 8.6.
- *8.19 Supóngase que se tienen k conjuntos, $S_1, S_2, \dots, S_k$, y cada uno con n números reales. Escribese un programa para listar todas las sumas de la forma $s_1 + s_2 + \dots + s_k$, donde s_i está en S_i , de acuerdo con algún orden de clasificación. ¿Cuál es la complejidad de tiempo del programa?
- 8.20 Supóngase que existe un arreglo clasificado de cadenas $s_1, s_2, \dots, s_n$. Escribese un programa para determinar si una cadena x dada es miembro de esta secuencia. ¿Cuál es la complejidad de tiempo del programa como una función de n y la longitud de x ?

Notas bibliográficas

Knuth [1973] es una referencia completa sobre métodos de clasificación. La clasificación rápida (*quicksort*) se debe a Hoare [1962] y las modificaciones posteriores fueron publicadas por Singleton [1969], y Frazer y McKellar [1970]. La clasificación por montículos (*heapsort*) fue descubierta por Williams [1964] y mejorada por Floyd [1964]. El árbol de decisión de la complejidad de la clasificación fue estudiado por Ford y Johnson [1959]. El algoritmo de selección lineal de la sección 8.7 es de Blum, Floyd, Pratt, Rivest y Tarjan [1972].

Shellsort se debe a Shell [1959] y su rendimiento ha sido analizado por Pratt [1979]. Véase Aho, Hopcroft y Ullman [1974] sobre una solución al ejercicio 8.9.

9

Técnicas de análisis
de algoritmos

¿Qué es un buen algoritmo? No es fácil responder a esta pregunta. Muchos criterios para un buen algoritmo incluyen cuestiones muy subjetivas como simplicidad, claridad y adecuación a los datos manejados. Una meta más objetiva, lo cual no significa que sea más importante, es la eficiencia en tiempo de ejecución. En la sección 1.5 se abordaron las técnicas básicas para establecer el tiempo de ejecución de programas simples. Sin embargo, en casos más complejos se requieren técnicas nuevas, como cuando los programas son recursivos. Este corto capítulo presenta algunas técnicas generales para resolver ecuaciones de recurrencia que se presentan en el análisis de los tiempos de ejecución de algoritmos recursivos.

9.1 Eficiencia de los algoritmos

Una forma de determinar la eficiencia del tiempo de ejecución de un algoritmo es programarlo y medir el tiempo que lleva la versión en particular, en un computador específico, para un conjunto seleccionado de entradas. Aunque es popular y útil, este enfoque tiene algunos problemas inherentes. Los tiempos de ejecución dependen no sólo del algoritmo base, sino también del conjunto de instrucciones del computador, de la calidad del compilador y de la destreza del programador. El programa también puede adaptarse para trabajar correctamente sobre el conjunto particular de entradas de prueba. Estas dependencias pueden ser muy notorias con un computador, un compilador, un programador o un conjunto de entradas de prueba distinto. Para superar esos inconvenientes, los expertos en computación han adoptado la complejidad de tiempo asintótica como una medida fundamental del rendimiento de un algoritmo. El término *eficiencia* se referirá a esta medida y, en especial, a la complejidad de tiempo en el peor caso (en contraposición al promedio).

Recuérdense del capítulo 1 las definiciones de $O(f(n))$ y $\Omega(f(n))$. La eficiencia o complejidad en el peor caso de un algoritmo es $O(f(n))$, o $f(n)$ para abreviar, si $O(f(n))$ es la función de n que da el máximo, en todas las entradas de longitud n , del número de pasos dados por el algoritmo en esas entradas. En otras palabras, existe alguna constante c tal que para una n suficientemente grande, $cf(n)$ es una cota superior del número de pasos dados por el algoritmo con cualquier entrada de longitud n .

En la aseveración de que «la eficiencia de un algoritmo dado es $f(n)$ », existe la implicación de que la eficiencia también es $\Omega(f(n))$, de forma que $f(n)$ es la función

de crecimiento más lenta de n que acota el tiempo de ejecución en el peor caso por arriba. Sin embargo, este último requisito no es parte de la definición de $O(f(n))$, y algunas veces no es posible asegurar que se tenga la cota de crecimiento superior más lenta.

Esta definición de eficiencia ignora factores constantes en el tiempo de ejecución y existen varias razones prácticas para ello. Primero, dado que la mayoría de los algoritmos están escritos en lenguajes de alto nivel, se deben describir en función de «pasos», los cuales emplean una cantidad constante de tiempo cuando se traducen al lenguaje de máquina de cualquier computador. Sin embargo, exactamente cuánto tiempo requiere un paso depende no sólo del paso mismo, sino del proceso de traducción y del conjunto de instrucciones de la máquina. Así, intentar tener más precisión que para decir que el tiempo de ejecución de un algoritmo es «del orden de $f(n)$ », esto es, $O(f(n))$, podría empantanar al usuario en detalles de máquinas específicas y sólo sería aplicable a esas máquinas.

Una segunda razón importante para tratar con la complejidad asintótica e ignorar factores constantes es que, más que estos factores, es la complejidad asintótica lo que determina para qué tamaño de entradas puede usarse el algoritmo para producir soluciones en un computador. En el capítulo 1 se analizó con detalle este aspecto. Sin embargo, es necesario tener cautela acerca de la posibilidad de que para problemas muy importantes, como los de clasificación, se pueda considerar adecuado analizar los algoritmos con tal detalle que sean factibles ciertas proposiciones como «el algoritmo A debe ejecutarse con el doble de rapidez que el algoritmo B en un computador típico».

Una segunda situación en la cual conviene desviarse de la noción de eficiencia en el peor caso ocurre cuando se conoce la distribución esperada de las entradas a un algoritmo. En estas situaciones, el análisis del caso promedio puede ser mucho más significativo que el análisis del peor caso. Por ejemplo, en el capítulo previo se analizó el tiempo de ejecución promedio de la clasificación rápida en el supuesto de que todas las permutaciones del orden de clasificación correcto tienen la misma probabilidad de presentarse como entradas.

9.2 Análisis de programas recursivos

En el capítulo 1 se mostró cómo analizar el tiempo de ejecución de un programa que no se llama a sí mismo en forma recursiva. El análisis de un programa recursivo es bastante distinto, y suele implicar la solución de una ecuación de diferencias. Las técnicas para la solución de ecuaciones de diferencias algunas veces son sutiles, y tienen una considerable semejanza con los métodos de solución de ecuaciones diferenciales, alguna de cuya terminología se utiliza.

Considérese el programa de clasificación presentado en la figura 9.1. Ahí, el procedimiento *mergesort* (clasificación por intercalación) toma una lista de longitud n como entrada, y devuelve una lista ordenada como salida. El procedimiento *combina*(L_1, L_2) toma como entrada las listas clasificadas L_1 y L_2 , y recorre cada una, elemento por elemento, desde el inicio. En cada paso, el mayor de los dos elementos

delanteros se borra de esta lista y se emite como salida. El resultado es una sola lista clasificada con los elementos de L_1 y L_2 . Los detalles de *combina* no tienen importancia alguna en este momento, puesto que este algoritmo de clasificación se analizará con detalle en el capítulo 11. Lo que importa es que el tiempo empleado por *combina* en listas de longitud $n/2$ es $O(n)$.

```

function mergesort ( L: LISTA; n: integer ) : LISTA;
{ L es una lista de longitud n. Se devuelve una versión clasificada de L.
  Se supone que n es una potencia de 2. }
var
    L1, L2: LIST
begin
    if n = 1 then
        return (L)
    else begin
        partir L en dos mitades, L1 y L2, cada una de longitud n/2;
        return (combina(mergesort(L1,n/2), mergesort(L2,n/2)))
    end
end; { mergesort }

```

Fig. 9.1. Procedimiento recursivo *mergesort*.

Sea $T(n)$ el tiempo de ejecución en el peor caso del procedimiento *mergesort* de la figura 9.1. Se escribe una ecuación de *recurrencia* (o *diferencias*) que acote $T(n)$ por arriba, como sigue

$$T(n) \leq \begin{cases} c_1 & \text{si } n = 1 \\ 2T(n/2) + c_2n & \text{si } n > 1 \end{cases} \quad (9.1)$$

El término c_1 en (9.1) representa el número constante de pasos dados cuando L tiene longitud 1. En el caso de que $n > 1$, el tiempo requerido por *mergesort* puede dividirse en dos partes. Las llamadas recursivas a *mergesort* con listas de longitud $n/2$ cada una llevan un tiempo $T(n/2)$, de aquí el término $2T(n/2)$. La segunda parte consiste en la prueba para descubrir que $n \neq 1$, la división de la lista L en dos partes iguales y el procedimiento *combina*. Esas tres operaciones requieren un tiempo que puede ser una constante, en el caso de la prueba, o ser proporcional a n al dividir y combinar. Así, la constante c_2 puede escogerse de modo que el término c_2n sea una cota superior del tiempo requerido por *mergesort* para hacer todo excepto las llamadas recursivas. La ecuación (9.1) representa todo lo anterior.

Obsérvese que (9.1) se aplica sólo cuando n es par, por lo que generará una cota superior en *forma cerrada* (esto es, como una fórmula para $T(n)$ que no implica ningún $T(m)$ para $m < n$) sólo cuando n es una potencia de 2. Sin embargo, aunque sólo se conozca $T(n)$ cuando n es una potencia de 2, se tiene una buena idea de $T(n)$ para toda n . En particular, para casi todos los algoritmos, se puede suponer que $T(n)$ está entre $T(2^i)$ y $T(2^{i+1})$ si n está entre 2^i y 2^{i+1} . Y con un poco más de esfuerzo

para encontrar la solución, se puede reemplazar el término $2T(n/2)$ de (9.1) por $T((n+1)/2) + T((n-1)/2)$ para $n > 1$ impar. Después, se podría resolver la nueva ecuación de diferencia para obtener una solución cerrada para toda n .

9.3 Resolución de ecuaciones de recurrencia

Existen tres enfoques distintos para resolver una ecuación de recurrencia.

1. Suponer una solución $f(n)$ y usar la recurrencia para mostrar que $T(n) \leq f(n)$. Algunas veces sólo se supone la forma de $f(n)$, dejando algunos parámetros sin especificar (por ejemplo, supóngase $f(n) = an^2$ para alguna a) y deduciendo valores adecuados para los parámetros al intentar demostrar que $T(n) \leq f(n)$ para toda n .
2. Utilizar la recurrencia misma para sustituir $m < n$ por cualquier $T(m)$ en la derecha, hasta que todos los términos $T(m)$ para $m > 1$ se hayan reemplazado por fórmulas que impliquen sólo $T(1)$. Como $T(1)$ siempre es constante, se tiene una fórmula para $T(n)$ en función de n y de algunas constantes. A esta fórmula se ha denominado «forma cerrada» para $T(n)$.
3. Emplear la solución general para ciertas ecuaciones de recurrencia de tipos comunes de esta sección o de otra (véanse las notas bibliográficas).

En esta sección se examinan los dos primeros métodos.

Suposición de una solución

Ejemplo 9.1. Considérese el método (1) aplicado a la ecuación (9.1). Supóngase que para alguna a , $T(n) = an \log n$. Al sustituir $n = 1$, se observa que esta suposición no funcionará, porque $an \log n$ tiene valor 0, independiente del valor de a . Así, se intenta a continuación $T(n) = an \log n + b$. Ahora, $n = 1$ requiere que $b \geq c_1$.

Para la inducción, se supone que

$$T(k) \leq ak \log k + b \quad (9.2)$$

para toda $k < n$ y se intenta establecer que

$$T(n) \leq an \log n + b$$

Para iniciar la demostración, se supone que $n \geq 2$. De (9.1),

$$T(n) \leq 2T(n/2) + c_2 n$$

De (9.2), con $k = n/2$, se obtiene

$$T(n) \leq 2[a \frac{n}{2} \log \frac{n}{2} + b] + c_2 n \quad (9.3)$$

$$\leq a n \log n - a n + c_2 n + 2b$$

$$\leq a n \log n + b$$

siempre que $a \geq c_2 + b$.

Así, $T(n) \leq a n \log n + b$ siempre y cuando se satisfagan dos restricciones: $b \geq c_1$ y $a \geq c_2 + b$. Por fortuna, existen valores que se pueden escoger para a que satisfacen ambas restricciones. Por ejemplo, al elegir $b = c_1$ y $a = c_1 + c_2$, por inducción sobre n , se concluye que para toda $n \geq 1$

$$T(n) \leq (c_1 + c_2)n \log n + c_1 \quad (9.4)$$

En otras palabras, $T(n)$ es $O(n \log n)$. $\square$

Son útiles dos observaciones acerca del ejemplo 9.1. Si se supone que $T(n)$ es $O(f(n))$, y si el intento de probar que $T(n) \leq c f(n)$ por inducción falla, se sigue que $T(n)$ no sea $O(f(n))$. De hecho, puede funcionar una hipótesis inductiva de la forma $T(n) \leq c f(n) - 1$.

En segundo lugar, aún no se ha determinado la tasa exacta de crecimiento asintótica para $f(n)$, aunque se ha demostrado que no es peor que $O(n \log n)$. Si se conjetura una solución más lenta, como $f(n) = a n$, o $f(n) = a n \log \log n$, no se puede demostrar la validez de $T(n) \leq f(n)$. La cuestión sólo se puede establecer concluyentemente examinando *mergesort* y demostrando que necesita realmente un tiempo $\Omega(n \log n)$; de hecho, requiere un tiempo proporcional a $n \log n$ en todas las entradas, no sólo en las peores entradas posibles. Se deja esta observación como ejercicio.

El ejemplo 9.1 expone una técnica general para demostrar que alguna función es una cota superior en el tiempo de ejecución de un algoritmo. Supóngase la ecuación de recurrencia

$$\begin{aligned} T(1) &= c \\ T(n) &\leq g(T(n/2), n), \text{ para } n > 1 \end{aligned} \quad (9.5)$$

Obsérvese que (9.5) generaliza (9.1), donde $g(x, y)$ es $2x + c_2 y$. También obsérvese que podrían imaginarse ecuaciones más generales que (9.5). Por ejemplo, la fórmula g puede comprender todos los $T(n-1)$, $T(n-2)$, ..., $T(1)$, y no sólo $T(n/2)$. También, es posible tener valores para $T(1)$, $T(2)$, ..., $T(k)$, y la recurrencia sólo sería aplicable para $n > k$. A manera de ejercicio, sería interesante considerar cómo resolver estas recurrencias más generales por el método (1), suponiendo una solución y verificándola.

Considérese ahora (9.5), en vez de sus generalizaciones. Supóngase una función $f(a_1, \dots, a_j, n)$, donde $a_1, \dots, a_j$ son parámetros, e inténtese demostrar por inducción en n que $T(n) \leq f(a_1, \dots, a_j, n)$. Por ejemplo, la suposición del ejemplo 9.1 fue $f(a_1, a_2, n) = a_1 n \log n + a_2$, pero se utilizaron a y b por a_1 y a_2 . Para corroborar que para algunos valores de $a_1, \dots, a_j$ se tiene $T(n) \leq f(a_1, \dots, a_j, n)$ para toda $n \geq 1$, se debe cumplir

$$\begin{aligned} f(a_1, \dots, a_j, 1) &\geq c \\ f(a_1, \dots, a_j, n) &\geq g(f(a_1, \dots, a_j, n/2), n) \end{aligned} \quad (9.6)$$

Esto es, por la hipótesis inductiva, se puede sustituir f por T en el lado derecho de la recurrencia (9.5) para obtener

$$T(n) \leq g(f(a_1, \dots, a_j, n/2), n) \quad (9.7)$$

Cuando se cumple la segunda línea de (9.6), se combina con (9.7) para demostrar que $T(n) \leq f(a_1, \dots, a_n, n)$, que es lo que se deseaba demostrar por inducción en n .

Por ejemplo, en el ejemplo 9.1, $g(x, y) = 2x + c_2y$ y $f(a_1, a_2, n) = a_1 n \log n + a_2$. Aquí se intenta satisfacer

$$\begin{aligned} f(a_1, a_2, 1) &= a_2 \geq c_1 \\ f(a_1, a_2, n) &= a_1 n \log n + a_2 \geq 2(a_1(\frac{n}{2})\log(\frac{n}{2}) + a_2) + c_2 n \end{aligned}$$

Como se analizó, $a_2 = c_1$ y $a_1 = c_1 + c_2$ son una elección satisfactoria.

Expansión de recurrencias

Si no se puede suponer una solución, o no existe la seguridad de tener la mejor cota en $T(n)$, se usa un método que, en principio, siempre es adecuado para la solución exacta de $T(n)$, aunque en la práctica a menudo surgen problemas al sumar series y hay que recurrir al cálculo de una cota superior en la suma. La idea general es tomar una recurrencia como (9.1), que indica que $T(n) \leq 2T(n/2) + c_2n$, y emplearla para obtener una cota en $T(n/2)$ sustituyendo $n/2$ por n . Esto es,

$$T(n/2) \leq 2T(n/4) + c_2n/2 \quad (9.8)$$

Al sustituir el lado derecho de (9.8) por $T(n/2)$ en (9.1), se obtiene

$$T(n) \leq 2(2T(n/4) + c_2n/2) + c_2n = 4T(n/4) + 2c_2n \quad (9.9)$$

Del mismo modo, se sustituye $n/4$ por n en (9.1) y se emplea para obtener una cota superior en $T(n/4)$. Esa cota, $2T(n/8) + c_2n/4$, se sustituye en el lado derecho de (9.9) para obtener

$$T(n) \leq 8T(n/8) + 3c_2n \quad (9.10)$$

Ahora es necesario percibir un patrón. Por inducción en i se obtiene la relación

$$T(n) \leq 2^i T(n/2^i) + ic_2n \quad (9.11)$$

para cualquier i . Si se supone que n es una potencia de 2, como 2^k , este proceso de expansión terminará tan pronto como se alcance $T(1)$ en el lado derecho de (9.11), lo que ocurrirá cuando $i = k$, quedando (9.11) como

$$T(n) \leq 2^k T(1) + kc_2n \quad (9.12)$$